Understanding the Potentially Confounding Effect of Test Suite Size in Test Effectiveness Evaluation

ZEYU LU, YANG WANG, YI RONG, YIFAN HUANG, XUAN WANG, and PENG ZHANG,
State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China
SHAN GAO, Independent Researcher, Hangzhou, China
MAOLIN SUN, YIBIAO YANG, YANHUI LI, LIN CHEN, and YUMING ZHOU, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

Background. Code coverage and mutation score serve as pivotal test effectiveness metrics used to assess a test suite's ability to uncover actual defects. However, prior research has produced inconsistent or even conflicting findings regarding their correlation with defect detection capability, particularly concerning the impact of test suite size. *Problem.* The extent of the potentially confounding effect of test suite size in test effectiveness evaluation context is not clear, nor is the method to remove the potentially confounding effect, or the influence of this removal on the performance of test suite optimization. *Objective.* Our goal is to deeply understand how test suite size affects the true relationship between test effectiveness metrics and a test suite's ability to detect actual defects. *Method.* We first employ statistical methods to examine the extent of the potentially confounding effect of test suite size in the context of test effectiveness evaluation. After that, we propose a linear regression-based method to remove the potentially confounding effect of test suite size. Finally, we empirically explore the impact of this removal method on test suite optimization. *Result.* Our experimental results, based on the Defects4J defect dataset, uncovers that: (1) the confounding effect of test suite size on the associations between test effectiveness metrics and defect detection capability in general exists; (2) the proposed linear regression-based method can effectively remove the confounding effect; and (3) after removing the confounding effect, mutation score demonstrates superior effectiveness in predicting test suite effectiveness, while statement coverage is the least effective metric. Furthermore, both coverage-based and mutation-based test suite reduction exhibit enhanced cost-effectiveness in defect detection, and there is a marginal improvement in the speed of defect detection for coverage-based test case prioritization.

This work is partially supported by the National Natural Science Foundation of China (62172205, 62172202, 62272221, 624B2067) and the Jiangsu Natural Science Foundation (BK20231402).
Authors' Contact Information: Zeyu Lu, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: zeyulu@smail.nju.edu.cn; Yang Wang, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: njuwy@smail.nju.edu.cn; Yi Rong, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: 952713875@qq.com; Yifan Huang, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: 502023330024@smail.nju.edu.cn; Xuan Wang, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: xuan_wang@smail.nju.edu.cn; Peng Zhang (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: dz1833034@smail.nju.edu.cn; Shan Gao, Independent Researcher, Hangzhou, China; e-mail: gaoshan_cs@outlook.com; Maolin Sun, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: merlin@smail.nju.edu.cn; Yibiao Yang, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: yangyibiao@nju.edu.cn; Yanhui Li, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: yanhuili@nju.edu.cn; Lin Chen, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: lchen@nju.edu.cn; Yuming Zhou (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: zhouyuming@nju.edu.cn.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2026/4-ART129
https://doi.org/10.1145/3748504

Conclusion. When using test effectiveness metrics to assess test suite effectiveness, it is crucial to eliminate the influence of test suite size.

CCS Concepts: • **Software and its engineering → Software testing and debugging**;

Additional Key Words and Phrases: Confounding Effect, Code Coverage, Mutation Score, Test Suite Size, Test Suite Effectiveness

ACM Reference format:
Zeyu Lu, Yang Wang, Yi Rong, Yifan Huang, Xuan Wang, Peng Zhang, Shan Gao, Maolin Sun, Yibiao Yang, Yanhui Li, Lin Chen, and Yuming Zhou. 2026. Understanding the Potentially Confounding Effect of Test Suite Size in Test Effectiveness Evaluation. *ACM Trans. Softw. Eng. Methodol.* 35, 5, Article 129 (April 2026), 54 pages. https://doi.org/10.1145/3748504

1 Introduction

In the field of software testing, test suites play a crucial role in assuring the quality of software. Evaluating the effectiveness of a test suite, defined as its capability to detect genuine defects in the **Software Under Test (SUT)** [43, 45, 108], is of paramount importance for various tasks, including gauging the adequacy of testing the SUT and guiding the development of new test cases. However, it is important to acknowledge that in practice, prior knowledge of the actual defects in SUT is often unavailable, making it impossible to directly quantify the effectiveness of a test suite in terms of defect detection. Consequently, we rely on the characteristics of a test suite to estimate its ability to detect defects. Two frequently employed surrogate metrics for evaluating test suite effectiveness are code coverage and mutation score. The former assesses the proportion of executed code elements, such as statements and branches, in relation to the total code elements when the test suite is executed. The latter gauges the percentage of "mutants" (variations of the original code intentionally altered with defects) effectively "killed" (i.e., identified) by the test suite out of the total mutants generated for testing. These test effectiveness metrics serve as indirect indicators of how well a test suite is likely to perform in terms of defect detection.

Before utilizing code coverage and mutation score as practical metrics for evaluating test suite effectiveness, it is crucial to establish their robust connection with defect detection. Over the years, many studies have delved into this connection. The majority of these investigations suggest a strong correlation between code coverage/mutation score and the effectiveness of a test suite in detecting defects [6, 32, 33, 35]. Furthermore, a subset of these studies has demonstrated that mutation score exhibits a stronger correlation with test suite effectiveness than code coverage does [8, 51], despite the relatively higher cost associated with mutation analysis [55, 73, 114, 117]. Consequently, there is a widely accepted acknowledgment that code coverage and mutation score can be employed to assess a test suite's capability in defect detection.

However, conducting a direct correlation analysis between test effectiveness metrics and defect detection without accounting for test suite size could lead to misleading conclusions about their true relationship [3, 31, 107]. The reason is that test suite size has a noteworthy influence on code coverage and mutation score. As the size of a test suite grows, both code coverage and mutation score tend to increase, and consequently, the test suite's defect detection ability also appears to improve. In essence, test suite size may act as a confounding variable that affects the link between test effectiveness metrics and defect detection capability. In light of this insight, recent studies [45, 77] have taken a stratification method to control for the confounding effect of test suite size in the analyses. Specifically, these studies first utilized a random selection strategy to generate pseudo test suites—artificial test suites constructed through random selection—of fixed sizes from

a large test pool. Then, these pseudo test suites are classified into different groups within which the test suite size is fixed at the same level. Finally, the correlation between test effectiveness metrics and defect detection was evaluated within each single group. Surprisingly, these studies have unveiled a substantial decline in the correlation between test effectiveness metrics and defect detection capability. For example, Inozemtseva and Holmes [45] and Papadakis et al. [77] initially identified a moderate-to-strong correlation between test effectiveness metrics and defect detection, without controlling for test suite size. However, upon controlling for test suite size, the correlation diminishes and can potentially become negligible. These findings raise questions about the true extent to which test effectiveness metrics such as code coverage and mutation score are correlated with defect detection capability.

Recently, a study by Chen et al. [17] has shed light on critical methodological flaws associated with the previously mentioned stratification method used to address the influence of test suite size. Chen et al. [17] emphasize that, in practical test suite creation, developers typically do not set the number of tests as a primary objective, nor should they. This insight suggests that generating pseudo test suites of fixed sizes from a large test pool should be avoided for analyzing the correlation between test effectiveness metrics and defect detection capability. Furthermore, their analysis reveals that this stratification method may lead to misinterpretations of observed correlations. The crux of the issue lies in the fact that the theoretical maximum correlation is contingent upon the test suite size and differs drastically across various test suite sizes. Hence, a low observed correlation within a specific test suite size group does not inherently indicate a weak relationship between test effectiveness metrics and defect detection. Consequently, a fundamental question arises: How can we effectively control the influence of test suite size to accurately establish the relationships between test effectiveness metrics and defect detection capability?

To tackle the aforementioned challenge, our study utilizes statistical methods [81] to examine how test suite size influences the correlation between test effectiveness metrics and defect detection, avoiding direct manipulation of test suite size. Following previous studies [6, 17, 96], we employ a greedy selection method to generate test suites of varying sizes. Specifically, tests are selected from the test pool one at a time using a greedy approach to incrementally increase code coverage or mutation score for a given criterion. Test selection ceases when predefined coverage or score levels are met. Subsequently, we apply a statistical confounding effect model to quantify the potential confounding effect of test suite size. This approach not only helps us identify the presence of any confounding effect from a statistical standpoint but also enables the analysis of the direction and magnitude of the effect, should it exist. Importantly, this method avoids the methodological flaw resulting from directly manipulating test suite size [17, 45, 56, 58, 71, 77, 108, 115], as it does not require the analyzed test suites to be of the same level of size. Following this, we propose a linear regression-based method to remove the potential confounding effect of test suite size. This allows us to further explore the real capability of code coverage and mutation score in predicting test suite effectiveness. Furthermore, we investigate the utility of this removal in test suite optimization, including test suite reduction and test case prioritization.

Our experimental results, based on the Defects4J defect dataset, reveal several key findings. First, test suite size generally has a confounding effect on the relationship between test effectiveness metrics and defect detection capability. Second, this confounding effect can be successfully removed using a linear regression-based method. Third, after eliminating these confounding effects, mutation score proves to be more effective in predicting test suite effectiveness compared to statement and branch coverage. Additionally, both coverage-based and mutation-based test suite reductions show enhanced cost-effectiveness in defect detection. There is also a marginal improvement in the speed of defect detection when using coverage-based test case prioritization. In summary, our findings suggest that mutant detection serves as a more reliable proxy metric for assessing test

suite effectiveness. Furthermore, removing the confounding effect of test suite size is crucial when using test effectiveness metrics to evaluate test suite effectiveness.

In summary, our study makes the following main contributions:

(1) We utilize statistical methods to examine the extent of the potentially confounding effect of test suite size on the link between test effectiveness metrics and defect detection capability. This not only allows us to investigate potential confounding effects with robust statistical evidence but also helps us overcome the conceptual pitfalls associated with generating fixed-size test suites using the random selection methodology commonly employed in previous studies. Consequently, we find that test suite size has a strong positive confounding effect, which leads to an overestimate on the true association of code coverage and mutation score with defect detection capability.
(2) We introduce a linear regression-based method to eliminate the potentially confounding effect of test suite size. By applying this method, we conduct a detailed analysis using clean, size-effect-free code coverage and mutation score to discern which particular adequacy criterion displays a stronger capability in predicting a test suite's defect detection ability. Our results reveal that, after removing the confounding effect, mutation score emerges as the most superior predictor of test effectiveness, while statement coverage ranks as the least effective.
(3) We investigate the practical value of removing the confounding effect of test suite size for test suite optimization. On the one hand, for test suite reduction, the use of the adjusted code coverage/mutation score helps strike a better balance between test cost and defect detection effectiveness. On the other hand, for test case prioritization, the use of the adjusted coverage-based test case prioritization leads to a slight improvement in the speed of defect detection.

The remainder of this article is organized as follows: In Section 2, we delve into related work. Section 3 introduces our **Logistic Regression (LR)**-based statistical model, which is employed for examining the confounding effect. Section 4 elucidates the data analysis techniques we employ. Section 5 provides an overview of the design of our experiments. In Section 6, we present the outcomes of our experiments. Section 7 presents extended experimental results and engages in a discussion about the implications of our findings as well as potential threats to validity. Finally, in Section 8, we conclude the article and outline the direction for future research.

2 Related Work

This section presents related work on evaluating test suite effectiveness and reviews papers that study the confounding effect in the field of software engineering, as well as the influence of test suite size on test suite effectiveness.

2.1 Evaluation of Test Suite Effectiveness

Evaluating the effectiveness of a test suite represents a significant area of research. The effectiveness of a test suite is characterized by its ability to detect real defects in the SUT [43, 45, 108]. However, determining the actual presence of defects in software beforehand is often uncertain, which poses a challenge in directly quantifying the effectiveness of the test suite. Consequently, various properties of a test suite are utilized to predict its defect detection effectiveness.

Code coverage, measuring the percentage of executed elements (such as statements and branches) in a test suite relative to the total elements, is a commonly used proxy for test suite effectiveness. It is evident that a test suite cannot detect defects in software modules it does not execute [106]. Covering a software module in the SUT is a necessary condition for detecting its defects. Generally,

higher code coverage suggests greater test suite effectiveness. Therefore, utilizing a test suite with high code coverage that encompasses all possible execution elements is essential for detecting potential defects in the SUT. Many past studies [6, 32, 33, 35, 39, 42, 57] have conducted analyses on the relationship between code coverage and defect detection capability, mostly utilizing a range of correlation analyses (such as Kendall rank correlation) and regression analyses (for example, R-squared). Several studies [6, 32, 33, 35] indicate the importance of code coverage in evaluating the effectiveness of a test suite.

The mutation score, which measures the percentage of killed mutants out of the total generated, serves as another common proxy for test suite effectiveness. Mutants are generated using mutation operators, which define rules for modifying a program to simulate potential errors a developer might make during software development [47]. The ability of a test suite to detect these mutants—thereby "killing" them—serves as an indicator of its effectiveness. Generally, a higher mutation score suggests a more effective test suite. Therefore, the effectiveness of a test suite can be enhanced by killing previously undetected mutants. Previous studies [8, 51] have compared the defect detection capabilities of code coverage and mutation score, finding that mutation score generally exhibits a higher ability to detect defects. However, to acquire a mutation score, the test suite must be executed against each generated mutant. Despite its superior defect detection capability, mutation score incurs a higher cost compared to code coverage [55, 73, 114, 117].

2.2 Influence of Test Suite Size

The term confounding refers to a situation in which a non-causal association between two variables is observed due to the influence of a third variable, i.e., the confounding variable or confounder [104]. The confounding variable may lead to a spurious correlation between two variables, which may falsely obscure or accentuate the relationship [52]. As confounding factors may impact the experimental results, it is necessary to control the confounding effects in experimental design. Baah et al. [7] employ matching to mitigate the confounding effects of data and control dependences for fault localization, and the empirical results indicate that their technique improves the effectiveness of statistical fault localization. In software defect prediction, class size is related to defect-proneness, as larger classes typically tend to be more defective than smaller classes. Previous studies have examined the confounding effect of class size on the associations between object-oriented metrics and defect-proneness [24, 120]. Besides, Zhou et al. [119] examined the confounding effect of class size on the associations between object-oriented metrics and change-proneness.

When evaluating the effectiveness of test suites, regardless of the test adequacy criteria used, larger sizes of test suites usually yield higher code coverage or mutation scores and are more effective in detecting defects in the SUT. Consequently, test suite size may act as a confounding factor, influencing both test effectiveness metrics and defect detection capability. Table 1 provides a summary of the primary findings from 10 prior studies examining the influence of test suite size. For each study, we present the source of test cases, method of test suite construction, processing of test suite size, test adequacy criteria, statistical methods, programming language of used subjects, and type of defects.

As shown in Table 1, the experimental settings vary across studies, and the conclusions are inconsistent. Several studies find that test suite size as a confounder impacts test suite effectiveness [45, 58, 71, 77]. However, other studies conclude that test suite size has almost no impact on test suite effectiveness [31, 32, 56]. Specifically, Chen et al. [17] claim that test suite size is not a confounding variable based on simple statistical analysis. These inconsistent conclusions may cause confusion about the impact of test suite size on test suite effectiveness. In this article, we avoid potential threats to validity in experimental settings pointed out in previous studies [17, 115]. We explore the impact of test suite size on its effectiveness, using statistical methods to model the confounding effect of test suite size on the correlation between test effectiveness metrics and defect detection capability.

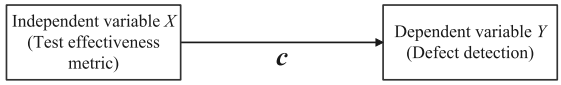
Table 1. Summary of Previous Studies on the Influence of Test Suite Size on Test Suite Effectiveness

Author(s) [Reference]	Year	Test Source	Test Suite Construction	Processing of Test Suite Size	Test Adequacy Criteria	Statistical Methods	Lang-uage	Defect Types	Summary of Primary Scientific Findings
Wong et al. [108]	1994	Automatically generated	Random selection	Manipulation but no stratification	Block	Spearman correlation, Kendall rank correlation, Partial rank correlation	C	Seeded defects	The correlation between effectiveness and block coverage is higher than that between effectiveness and size.
Andrews et al. [6]	2006	Existing test suites	Greedy selection, Random selection	No manipulation, Manipulation but no stratification	Block, decision, c-use, p-use	R-squared	C	Mutants	Both size and coverage play complementary roles in explaining defect detection and are strongly correlated.
Namin et al. [71]	2009	Existing test suites	Random selection	Manipulation but no stratification	Block, decision, c-use, p-use	ANCOVA, PCA, Pearson correlation, Kendall rank correlation, adjusted R-squared	C	Mutants	Both size and coverage independently influence test suite effectiveness. There is a nonlinear relationship among the three variables of size, coverage, and effectiveness.
Gligoric et al. [32]	2013	Existing test suites, automatically generated	Coverage-based selection, Random selection	No manipulation, Manipulation but no stratification	Statement, branch, IMP, AIMP, PCT	Kendall rank correlation, R-squared	C, Java	Mutants	Multiple criteria effectively predict mutation scores. Branch coverage performs best to predict mutation scores. Using size alone to predict mutation score is extremely ineffective. Adding size as an additional variable in regressions does not change the general correlation results.
Inozemtseva and Holmes [45]	2014	Existing test suites	Random selection	Manipulation and stratification by same size	Statement, decision, modified condition	Adjusted R-squared, Kendall rank correlation, Pearson correlation	Java	Mutants	There is a moderate-to-very high correlation between test suite effectiveness and test suite size. There is a moderate-to-high correlation between effectiveness and coverage when the influence of test suite size is ignored. There is a low-to-moderate correlation between coverage and effectiveness when test suite size is controlled for.
Kochhar et al. [58]	2015	Automatically generated	Random selection	Manipulation but no stratification	Statement, branch	Point biserial correlation	Java	Real defects	Test suite size has a weak-to-strong correlation with test suite effectiveness. Code coverage has a moderate or strong correlation to the effectiveness of a test suite.
Gay [31]	2017	Automatically generated	Each test suite is used for analysis without selecting	No manipulation	BC, DBC, LC, EC, MC, MNEC, OC, WMMC	Treatment Learning	Java	Real defects	The strongest indicator of test suite effectiveness is a high level of code coverage. Specifically, branch coverage is the most effective. Larger test suites are not necessarily more effective at detecting real faults.
Papadakis et al. [77]	2018	Existing test suites, automatically generated	Random selection	Manipulation and stratification by same size	Mutation	Pseudo R-squared, Kendall rank correlation, Pearson correlation	C, Java	Real defects	Test suite size and mutation score independently influence test effectiveness. All correlations between mutation score and real defect detection are weak when controlling for test suite size. Mutants provide good guidance for improving the defect detection ability of test suites.
Kim et al. [56]	2020	Existing test suites, automatically generated	Random selection	Manipulation but no stratification	Mutation	Pseudo R-squared, Point biserial correlation	Java	Real defects	There is a high and independent correlation between the mutation score and defect detection, independently of the test suite size, at a fine-level granularity.
Chen et al. [17]	2020	Existing test suites	Adequacy-based test selection, test suite reduction	No manipulation	Code coverage, mutation	-	Java	Real defects	Test suite size is neither a confounding variable nor an independent variable that should be experimentally manipulated. Mutation-based test selection is most effective after coverage has exhausted its usefulness.
Our study	2023	Existing test suites	Greedy selection	No manipulation	Statement, branch, mutation	Confounding Effect model	Java	Real defects	The association between test effectiveness metrics and defect detection capability is generally confounded by the test suite size, leading to an overestimation of this association.

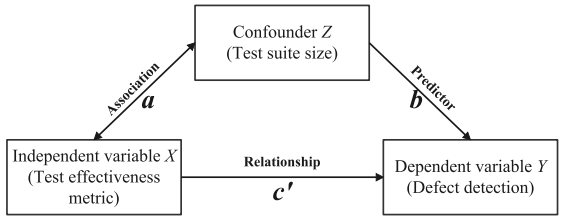
In the *Test Adequacy Criteria* column, c-use stands for computational use, and p-use for predicate use. The abbreviations IMP, AIMP, and PCT refer to intra-method paths, acyclic intra-method paths, and predicate-complete test coverage, respectively. Furthermore, BC, DBC, LC, EC, MC, MNEC, OC, and WMMC represent branch coverage, direct branch coverage, line coverage, exception coverage, method coverage, method coverage with no top-level exceptions, output coverage, and weak mutation coverage, respectively.

3 The Confounding Model

As shown in Table 1, there are *inconsistent* conclusions about whether test suite size impacts test suite effectiveness and whether test suite size is a confounding variable. In this article, we aim to conduct an in-depth study on the potentially confounding effect of test suite size and explore the true relationship between test effectiveness metrics and defect detection capability. In this context, the test effectiveness metric serves as the independent variable (X), while defect detection is the dependent variable (Y).



(a) Total Effect of X on Y



(b) The Confounding Model

Fig. 1. The confounding effect model, in which the confounder Z may falsely obscure or accentuate the relationship between X and Y.

3.1 Conceptual Model

Figure 1(b) represents the confounding model [25], illustrating the relationship between the three variables X, Y, and Z. The confounding variable Z relates to both the independent variable X and the dependent variable Y.

Identification of Confounder. According to Farajollahi et al. [25], and consistent with many existing studies [46, 59, 70, 89, 98, 105], a potential confounder can be identified if it satisfies the following three criteria:

(1) It should be related to the independent variable.
(2) It should be a risk factor for or a predictor of the dependent variable.
(3) It should not be part of the causal pathway from the independent variable to the dependent variable.

Relationship between the Variables. According to the three criteria for identifying a confounder [25, 46, 59, 70, 89, 98, 105], the relationship between Z and X, or between Z and Y, does not necessarily have to be causal.

(1) *The relationship between the confounder and the independent variable.* Kumar and Baqui [70] present the criterion that the confounder must be either causally or non-causally associated with the independent variable. Many existing studies [25, 46, 59, 63, 69, 89, 98, 105, 111] require only that the confounder is associated with the independent variable. For example, Mehio-Sibai et al. [69] provide an example in which smoking (Z) acts as a confounder in the relationship between alcohol (X) and cardiovascular disease (Y). In this example, smoking does not cause alcohol consumption.

(2) *The relationship between the confounder and the dependent variable.* Van Stralen et al. [105] present the criterion that, for a variable to be considered a potential confounder, the variable needs to be associated with the dependent variable. For the second criterion presented by Farajollahi et al. [25], a potential confounder should be a risk factor for or a predictor of the dependent variable. Specifically, a risk factor or a predictor does not need to be causal [38]. Skelly et al. [98] state that a potential confounding factor is predictive of the dependent variable, and although a potential confounding factor may be causative, it might not be. Kyriacou and Lewis [59] state that a confounder must be an independent risk factor for the dependent variable, either a causal factor

or a surrogate for a causal factor. Jager et al. [46] present the criterion that a potential confounder must be associated with the dependent variable, that is, it should be a risk factor for the dependent variable.

Concrete Examples. Many examples of identifying potential confounders have been provided in published studies [25, 46, 59, 63, 69, 70, 89, 98, 105].

Van Stralen et al. [105] discussed an example examining whether obesity (Z) is a confounder in the relationship between hypertension (X) and **End-Stage Renal Disease (ESRD**, Y). Criterion 1 is fulfilled because obesity is associated with hypertension (many obese patients develop hypertension). Criterion 2 is satisfied since obesity is a factor related to ESRD (many obese patients develop ESRD). Criterion 3 is fulfilled because hypertension does not cause obesity, meaning that obesity is not in the causal pathway from hypertension to ESRD.

Satyarup et al. [89] presented an example in which age (Z) is a confounder in the association between physical activity (X) and lung cancer (Y). Age is associated with physical activity, as older individuals tend to be more physically inactive (criterion 1 fulfilled). Age is also associated with lung cancer, as older adults are at higher risk of developing it (criterion 2 satisfied). Moreover, age is not in the causal pathway linking the independent and dependent variables (criterion 3 fulfilled).

Relationship between Test Effectiveness Metrics, Test Suite Size, and Defect Detection. Many previous studies [6, 39, 45, 77] consider test suite size to be a confounder. Test suite size relates to both code coverage/mutation score and defect detection. Test suite size satisfies the three criteria that a confounder should meet:

(1) A large test suite size tends to have high code coverage or mutation score. High code coverage or mutation score also tends to be associated with a large test suite size. The relationship between test effectiveness metrics and test suite size is associative.
(2) Previous studies [6, 45, 58, 71, 77, 108] have shown that test suite size influences defect detection. In test suite reduction, while code coverage or mutation score remains unchanged, reducing the test suite size to eliminate redundant test cases can lead to a decrease in defect detection capability, although the impact may vary in significance [86, 87, 95, 109, 110, 116]. The relationship between test suite size and defect detection is associative. Test suite size is a predictor, a surrogate for a causal factor, and could be a determinant of defect detection.
(3) High code coverage or mutation score does not necessarily lead to a large test suite size, which in turn leads to high defect detection capability. A causal pathway from test effectiveness metrics to test suite size and then to defect detection cannot be established.

3.2 Quantification of Confounding

We use a statistical model to quantify the relationship between a given test effectiveness metric, test suite size, and defect detection. Specifically, the dependent variable is dichotomous, with only two values to designate whether a test suite detects defects. In the Discussion section, the dependent variable is modeled as a continuous variable. We primarily use a dichotomous dependent variable due to the isolated real defects. We use the statistical model to explore the confounding effect of test suite size Z.

Given that Y is a dichotomous variable, we employ LR, *a non-linear model*, to analyze its relationship with independent variables [66]. The general LR equation is:

$$ln\left(\frac{p(Y=1)}{1-p(Y=1)}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \varepsilon. \quad (1)$$

In Equation (1), the log of odds ratio in LR is on the left side, and a linear regression model is on the right side. To simplify the notation, let Y^* denote $ln(\frac{p(Y=1)}{1-p(Y=1)})$.

Figure 1(a) presents the relationship between the independent variable X and the dependent variable Y. The equation between X and Y is:

$$Y^* = i_{Y^{**}} + cX + \varepsilon_{Y^{**}}, \tag{2}$$

where $i_{Y^{**}}$ is the intercept, c is the regression coefficient, and $\varepsilon_{Y^{**}}$ is the error term.

The two equations corresponding to Figure 1(b) are:

$$X = i_X + aZ + \varepsilon_X, \tag{3}$$

$$Y^* = i_{Y^*} + c'X + bZ + \varepsilon_{Y^*}, \tag{4}$$

where i_X and i_{Y^*} are the intercepts, a, c', and b are the regression coefficients, ε_Z and ε_{Y^*} are the error terms.

Let $\hat{c}$, $\hat{a}$, $\hat{c}'$, and $\hat{b}$ represent the estimates of these regression coefficients. These estimates can be obtained using ordinary least squares estimation or maximum likelihood estimation.

In Equation (2), the coefficient c estimates the *total effect* of X on Y. In Equation (4), the coefficient c' estimates the *direct effect* of X on Y, taking into account the third variable.

Based on Equation (3), the regression of Z on X is represented by:

$$Z = \frac{1}{a}(X - i_X - \varepsilon_X). \tag{5}$$

The regression of Z on X is used to calculate the association between X and Z.

Equation (4) can be expanded by including the regression of Z on X:

$$\begin{aligned} Y^* &= i_{Y^*} + c'X + bZ + \varepsilon_{Y^*} \\ &= i_{Y^*} - \frac{b}{a}i_X + \left(c' + \frac{b}{a}\right)X - \frac{b}{a}\varepsilon_X + \varepsilon_{Y^*}. \end{aligned} \tag{6}$$

Combining Equation (2) can yield:

$$c = c' + \frac{b}{a}. \tag{7}$$

The $\frac{b}{a}$ represents the bias due to confounding, while c differs from c' due to this confounding bias. There are two approaches to quantifying the confounding effect of Z in the relationship between X on Y [67]. The first approach is $\frac{b}{a}$, which is the product of the parameters $\frac{1}{a}$ and b. The other is $c - c'$, which is the difference between the parameters c and c'. If the scaling across Equations (2) and (4) is equalized by standardizing the regression coefficients, then the indirect effects $\frac{b}{a}$ and $c - c'$ will approximately be equal [66].

The confounding model does not require fixing one of the three variables and allows for the analysis of the effect of test suite size on both test effectiveness metrics and defect detection. Test suite reduction requires reduced and unreduced test suites to have the same test effectiveness metrics, the confounding model adopted in this article is not limited to scenarios where the test suite size and defect detection vary while the test effectiveness metrics remain constant. For previous study [17] that utilize test suites constructed by random selection, which are of the same size as those constructed based on test effectiveness metrics, to analyze the influence of test suite size on defect detection, the confounding model addresses the limitation of only analyzing the influence of test suite size on defect detection.

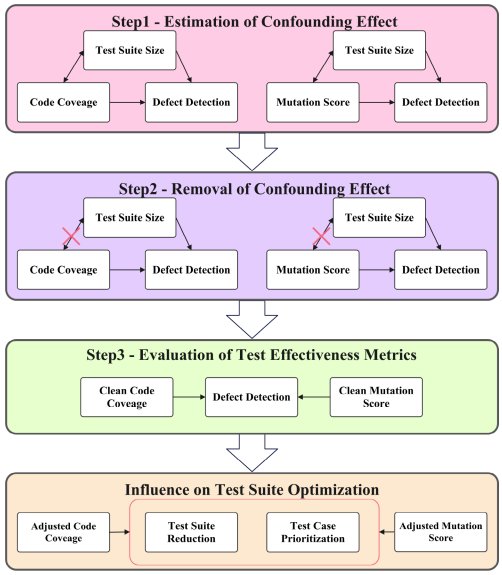


Fig. 2. Overview of the data analysis methods.

4 Data Analysis Method

Our study aims to investigate the confounding effect of test suite size on the relationship between test effectiveness metrics and defect detection capability. In the previous section, we introduced the confounding model. This section details the data analysis methods employed for further investigation of this effect.

4.1 Research Questions

Figure 2 illustrates the three main analysis steps: estimation of confounding effect, removal of confounding effect, and evaluation of test effectiveness metrics. Based on the confounding model, we assess the significance of test suite size as a confounding effect. This is intended to answer the first research question:

RQ1: *Does test suite size have a confounding effect on the relationship between test effectiveness metrics and defect detection capability?*

Upon testing the statistical significance of the confounding effect with the confounding effect model, if it is found to be significant, we aim to remove the confounding effect of test suite size. This leads to our second research question:

RQ2: *If test suite size has a confounding effect, how to remove the confounding effect of test suite size on the relationship between test effectiveness metrics and defect detection capability?*

Once we have successfully removed the confounding effect of test suite size, we obtain the *clean* code coverage and *clean* mutation score, which eliminates the influence of the confounder. We can then explore the real relationship between test effectiveness metrics and defect detection

capability based on the clean code coverage and clean mutation score. We formulate our third research question:

RQ3: *If test suite size has a confounding effect, what is the real relationship between test effectiveness metrics and defect detection capability?*

As most related work [6, 17, 31, 32, 45, 56, 58, 71, 77, 108] uses the number of test cases as the measure for test suite size, we primarily adopted this measure to facilitate comparisons with previous studies. We use two alternative test suite size measures: the number of lines of test code and the number of assertions. Following the three main steps, we first assess whether the confounding effect is present. If it is, we then analyze the true relationship between test effectiveness metrics and defect detection after removing the confounding effect. This leads to our fourth research question:

RQ4: *When using alternative measures of test suite size, does the confounding effect exist, and if that is the case, what is the true relationship between test effectiveness metrics and defect detection capability?*

We primarily model the dependent variable as a dichotomous variable in the research questions above. To enhance generalizability, we model the dependent variable as a continuous variable by assuming the mutation score as a proxy for defect detection. We then compare which code coverage measure better predicts the mutation score. This leads to our fifth research question, for which we follow the three-step analysis approach:

RQ5: *Assuming the mutation score is a proxy for defect detection, which code coverage measure better predicts the mutation score?*

Based on the confounding removal method, the clean test effectiveness metrics are the size-adjusted code coverage or mutation score. Inspired by this, we want to investigate the influence of size-adjusted code coverage or mutation score on test suite optimization. To this end, we first design test suite reduction and test case prioritization strategies that iteratively expand a test suite, allowing raw code coverage and mutation score to be adjusted for test suite size. Then, we examine whether there are benefits in terms of defect detection loss for reduced test suites and defect detection speed for prioritized test suites using these metrics. This leads to our sixth research question:

RQ6: *How do coverage- and mutation-based test suite optimization approaches perform when using size-adjusted code coverage and mutation score?*

We divide this research question into two sub-questions: RQ6.1 investigates test suite reduction, and RQ6.2 focuses on test case prioritization.

4.2 Estimation of Confounding Effect

For the two approaches to quantifying the confounding effect, we use the test based on $\frac{b}{a}$ rather than on c-c' as suggested by Rijnhart et al. [84]. According to MacKinnon et al. [67], the mediation effect and the confounding effect can be analyzed using the same statistical methods, although there are conceptual differences between these two types of third-variable effects. The statistical identity between mediation and confounding lies in the fact that the relationship between the independent variable (X) and the third variable (Z) can be calculated by the regression of Z on X. The difference between the two types of third-variable effects is that X causes Z in the mediation model, thus there is an arrow from X to Z. In contrast, the confounding model has an arrow from

Z to X, showing that Z influences X in the associative relationship between X and Z, which is in the opposite direction of the arrow in the mediation model. There are studies [69, 119, 120] adopt statistical methods used in mediation analysis for confounding analysis. Therefore, we adopt the estimates of the mediation effect to estimate the confounding effect. We also select the effect size used in the mediation analysis to measure the indirect effect of the confounder.

To test the confounding effect, one of the commonly used tests is the Sobel test [100, 101]. However, the Sobel test assumes that the estimator $\frac{b}{a}$ of the confounding effect has a normal distribution. In this experiment, most data do not follow a normal distribution, which violates the inferential assumption of the Sobel test. The bootstrap method is suitable for the situation where the data are not normally distributed [66]. It is a resampling method. Assuming the sample is of size n, the bootstrap method randomly samples data of size n with replacement from the original dataset. This process is repeated many times, such as 1,000 or 5,000 times or more. These samples are then used to compute confidence limits. Besides, the bootstrap method is a widely used and recommended method [66, 68], and it is more powerful than the Sobel test [41].

There are mainly two bootstrap methods: the percentile bootstrap and the bias-corrected bootstrap. In this article, we use the percentile bootstrap method to test the confounding effect. The percentile bootstrap method is a good compromise between accuracy and power, and it has more accurate Type I error rates than the bias-corrected bootstrap method [41]. In the percentile bootstrap method, the null hypothesis is that $\frac{b}{a} = 0$. If the confidence interval includes zero, we fail to reject the null hypothesis, which means that test suite size Z does not have a confounding effect. Otherwise, we reject the null hypothesis, which means that the confounding effect is significant and Z has a confounding effect. We set the number of bootstrap samples to 5,000, which is recommended by Preacher and Hayes [82] and used by Hayes [40]. We utilize the PROCESS macro by Hayes [40] to conduct confounding analysis. We do not standardize the regression coefficients, as Rijnhart et al. [84] suggest that standardizing the coefficients prior to estimating the confounding effect does not enhance the performance of the estimates of this effect.

In addition, we use the $\frac{\frac{b}{a}}{\frac{b}{a}+c'} = \frac{b}{b+ac'}$ estimate of the proportion confounded as an effect size measure. This estimate is calculated by dividing the indirect effect b/a by the sum of the indirect effect b/a and the direct effect c'. The use of the $b/(b+ac')$ estimate of the proportion confounded is recommended by Rijnhart et al. [84]. The proportion confounded can also be calculated as dividing the indirect effect b/a by the total effect c, i.e., b/ac, and dividing the subtraction of the direct effect c' from the total effect c by the total effect c, i.e., $1 - (c'/c)$. Rijnhart et al. [84] compared the performance of the three approaches and observed the lowest bias and highest efficiency for $b/(b + ac')$.

If Z has a confounding effect, we judge the direction of the confounding effect. The direction of the confounder can be either *positive* or *negative*. By identifying the direction, we can further judge whether the confounder leads to an *overestimate* or *underestimate* of the original relationship.

We apply the method proposed by Mehio-Sibai et al. [69] to identify whether test suite size is a positive confounder or a negative confounder. Their method uses the sign of the b/a estimator and the c' estimate. A positive sign indicates a positive association, while a negative sign indicates a negative association. If the signs of b/a and c' are the same, the direction of the confounder is positive. In this case, the effect of the confounder can lead to an overestimate of the crude relation. On the contrary, if the sign of b/a is different from the sign of c', the direction of the confounder is negative, and the negative confounder leads to an underestimate of the primary relation.

4.3 Removal of Confounding Effect

If the confounding effect is significant, the association between independent variable X and dependent variable Y is influenced by the confounder Z. In this article, we focus on the true relationship between X and Y. Therefore, the independent variable X needs to be adjusted to be free from the influence of the confounder Z. We use a linear regression-based method [23] to remove the confounding effect of test suite size on the association between test effectiveness metrics and defect detection capability. For the independent variable X and the confounder Z, we have:

$$X = i_X + a_r Z + \varepsilon_X, \tag{8}$$

where i_X is the intercept, a_r is the regression coefficient, and ε_X is the error term.

In Equation (8), X can be explained by Z, which represents the part of X that is predictable from Z. The parameter ε_X represents the part of X that cannot be explained by Z, which is independent of Z. In theory, the error term ε_X is free from the influence of Z. To remove the confounding effect, we subtract the predicted $\hat{X}$ from X. Let X_r denote the difference between X and $\hat{X}$. X_r is the estimate of the error term ε_X and is called the residual. We have:

$$\begin{aligned} X_r &= X - \hat{X} \\ &= X - \hat{a_r} Z - \hat{i_X}. \end{aligned} \tag{9}$$

Here, X represents the *original* code coverage/*original* mutation score, and X_r represents the *clean* code coverage/*clean* mutation score.

After removing the confounding effect, we apply the bootstrap method to test whether Z has a confounding effect on the relationship between X_r and Y. If we fail to reject the null hypothesis, this suggests that we have successfully removed the confounding effect. If we reject the null hypothesis, it indicates a failure to remove this effect.

4.4 Evaluation of Test Effectiveness Metrics

After removing the confounding effect, we obtain the clean code coverage/mutation score X_r. We analyze the relationship between X_r and Y and use X_r to construct the test suite effectiveness prediction model. In the confounding model, we use LR to analyze the relationship between X, Z, and Y. Therefore, we also adopt LR to characterize the relationship between X/X_r and Y, as well as the relationship between X/X_r, Z, and Y.

In this article, Y is a dichotomous variable. To mitigate the threat of relying on one predictive model, we also use other machine learning models to predict test suite effectiveness. Specifically, we train different models based on X/X_r and then measure the predictive ability of these models. We also incorporate Z to construct predictive models. Besides LR, we use **Decision Tree (DT)**, **K-Nearest Neighbors (KNN)**, **Naive Bayes (NB)**, **Multi-Layer Perceptron (MLP)**, **Support Vector Machine (SVM)**, and **Random Forest (RF)**, all of which are commonly used classification models, to classify test suites. Since the prediction performance changes with different hyperparameters, we leverage **Grid Search (GS)** to find the optimal hyperparameters of each model.

The main purpose of predicting test suite effectiveness is to explore the relationship between test effectiveness metrics, test suite size, and defect detection capability. Specifically, we use prediction performance to explore the extent to which code coverage/mutation score explains defect detection capability and whether test suite size explains defect detection capability.

For binary classification, the classification evaluation can be based on the confusion matrix. Table 2 presents the confusion matrix. The row of the table represents the actual class, while the column represents the predicted class. From this confusion matrix, **True Positive (TP)** and

Table 2. Confusion Matrix for Binary Classification

	Classified Positive	Classified Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

True Negative (TN) are the number of positive and negative classes that are correctly classified, respectively. **False Positive (FP)** and **False Negative (FN)** are the number of misclassified negative and positive classes, respectively.

To assess the classification performance of the seven classification models, we choose the F1-score as the evaluation metric. The F1-score is the harmonic mean of the *precision* and *recall*:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}, \tag{10}$$

$$Precision = \frac{TP}{TP + FP}, \tag{11}$$

$$Recall = \frac{TP}{TP + FN}. \tag{12}$$

As Chen et al. [17] point out, defect detection often suffers from class imbalance. Therefore, we also adopt the Geometric Mean (G-mean), which balances between FP and FN rates [4], as an evaluation metric of predictive models. The G-mean is the squared root of the product of the *sensitivity* and *specificity*:

$$\begin{aligned} G\text{-}mean &= \sqrt{Sensitivity \times Specificity} \\ &= \sqrt{\frac{TP}{TP + FN} \times \frac{TN}{TN + FP}}. \end{aligned} \tag{13}$$

4.5 Influence on Test Suite Optimization

The confounding effect is analyzed using a large number of test suites. Unlike confounding analysis, test suite optimization is performed for each individual test suite. In this section, we explore the impact of coverage-based and mutation-based test suite optimization using size-adjusted code coverage or mutation score. Specifically, we examine coverage-based and mutation-based test suite reduction and test case prioritization approaches that iteratively expand a test suite, allowing raw code coverage or mutation score to be adjusted for test suite size.

4.5.1 Test Suite Reduction. Test suite reduction aims to reduce the number of test cases in a test suite while still satisfying all the requirements covered by the unreduced test suite. There are a lot of research on test suite reduction based on code coverage or mutation score [9, 16, 37, 74, 94]. However, there are inconsistent conclusions in the previous work on the defect detection capability of reduced test suites. As shown in Table 3, several studies conclude that test suite reduction can reduce test suite size without significantly reducing the defect detection capability [109, 110, 116], while other studies find that test suite reduction significantly reduce defect detection capability [86, 87, 95].

Given a test suite $T = \{t_1, t_2, \dots, t_n\}$ and a set of test requirements $R = \{r_1, r_2, \dots, r_m\}$, the problem of test suite reduction is to find a subset $T' \subseteq$ T that satisfies all requirements that are satisfied by T. Let $S(T, R)$ denote the set of test requirements R satisfied by test suite T.

In this article, we implement test suite reduction using the widely adopted greedy algorithm [94]. The algorithm starts with an empty set $T' = \varnothing$. Then, it repeatedly selects test case t_i that satisfies

Table 3. Summary of Previous Studies on the Relationship between Test Suite Reduction and Defect Detection Capability Reduction

Author(s) [Reference]	Year	Test Adequacy Criteria	Statistical Methods	Language	Defect Types	Summary of Primary Scientific Findings	
Wong et al. [109]	1995	Block, all-uses	-	C	Seeded defects	The decrease in defect detection capability for the minimized test suites is slight even when the reduction in test suite size is great.	Test suite reduction can reduce test suite size without significantly reducing the defect detection capability.
Wong et al. [110]	1997	Block	-	C	Real defects	Minimized test suites have a significant reduction in test suite size but with almost the same defect detection effectiveness over the non-minimized test suites.	
Zhang et al. [116]	2011	Method, statement	ANOVA	Java	Seeded defects, mutants	The greedy, Harrold et al.'s heuristic, GRE heuristic, and ILP techniques can effectively reduce test suites without substantially reducing their defect detection capability.	
Rothermel et al. [86]	1998	Edge	Pearson correlation	C	Seeded defects	The defect detection capabilities of test suites can be severely compromised by minimization.	Test suite reduction significantly reduce defect detection capability.
Rothermel et al. [87]	2002	Edge	-	C	Seeded defects, real defects	The defect detection capabilities of test suites can be severely compromised by test suite reductions.	
Shi et al. [95]	2018	Line, mutation	Kendall rank correlation, R Squared	Java	Real defects, mutants	The Failed-Build Detection Loss (FBDL) is quite high, and traditional TSR metrics are not good predictors of FBDL.	
Chen et al. [17]	2020	Coverage, mutation	-	Java	Real defects	Test suite reduction based on mutants maintains defect detection capability, while test suite reduction based on coverage leads to a substantial loss of defect detection capability.	

the maximum unsatisfied test requirements, namely $\forall t_j \in (T - T'), |\ S(t_j, r)| \leq |S(t_i, r)|$, where $r = R - S(T', R')$ and $j \neq i$. The algorithm then adds t_i into T', i.e., $T' = T' \cup t_i$. If all requirements satisfied by T have been satisfied by T', i.e., $|\ S(T', R)| = |\ S(T, R)|$, the algorithm terminates and returns T'. Since the greedy algorithm adopts the current best strategy, it provides a near-optimal solution [53].

The performance of the test suite reduction approaches is evaluated using the following evaluation metrics, which are widely used in previous studies [86, 94, 116]:

(1) **Percentage of Test Suite Reduction (PTSR):** We quantify the reduction in the number of test cases achieved by the reduction approach:

$$PTSR = \frac{|T| - |T'|}{|T|} \times 100\%, \tag{14}$$

where $|T|$ and $|T'|$ are the number of test cases in T and T', respectively.

(2) **Fault Detection Loss (FDL):** We assess the FDL incurred by the reduction approach:

$$FDL = \frac{|F| - |F'|}{|F|}, \tag{15}$$

where $|F|$ and $|F'|$ are the number of faults detected by T and T', respectively.

4.5.2 Test Case Prioritization. Given a test suite T, let PT be the set of permutations of T, and let f denote the function $PT \rightarrow \mathbb{R}$, which maps PT to real numbers. The problem of test case prioritization is to find $T' \in PT$ such that $(\forall T'')(T'' \in PT)(T'' \neq T')(f(T') \geq f(T''))$ [88]. Test case prioritization aims to order test cases for more rapid defect detection. Many existing studies on this topic are based on code coverage or mutation score [65, 88, 97, 102].

In this article, we adopt the additional approach [88, 97] to prioritize a test suite T. The additional approach has been widely recognized as a state-of-the-art technique since it is simple but effective [18]. For statement coverage and branch coverage, the approach iteratively selects test case t_i which covers maximum number of uncovered statements and branches, and the process repeats until all

statements and branches have been covered. Similarly, for mutation score, the approach iteratively selects test case t_i that maximizes the number of additionally killed mutants. In the process of test case prioritization, the approach may reach the cases where the remaining test cases cannot add additional code coverage or kill additional mutants. Similar to previous study [88], we reset the code coverage/mutation score to their initial values and reapply the additional approach for the remaining test cases.

To assess the effectiveness of test case prioritization approaches, the most widely used metric is **Average Percentage of Faults Detected (APFD)** [54]:

$$APFD = 1 - \frac{TF_1 + TF_2 + \cdots + TF_r}{nr} + \frac{1}{2n}, \tag{16}$$

where r represents the number of defects, n denotes the number of test cases in T, and TF_i is the ranking of the test case in the prioritized test suite that first detects the ith defect.

However, there exists difference in terms of the execution cost of each test case, which is ignored by APFD. In this article, we adopt $APFD_C$ (Cost-cognizant Average Percentage of Fault Detection) to measure the performance of test case prioritization based on original code coverage/mutation score and clean code coverage/mutation score. In practice, it is difficult to know the severity of each defect in advance. Similar to pervious work [112], we use a simplified $APFD_C$, which assigns a severity of 1 to each defect, to assess prioritization performance. The simplified $APFD_C$ is defined as follows:

$$APFD_C(simplified) = \frac{\sum_{j=1}^{m}(\sum_{i=TF_j}^{n} t_i - \frac{1}{2} \times t_{TF_j})}{\sum_{j=1}^{n} t_j \times m}, \tag{17}$$

where n is the test suite size, m refers to the number of defects detected by the test suite, TF_j is the position of the first test case that detects defect j in the prioritized test suite, and t_j is the test cost of the jth test case in the prioritized test suite.

In this article, we consider three types of test costs for test case t: **Lines of Code (LOC)**, the number of assertions, and execution time. The test cost of test case t is higher if it has more LOC, contains greater number of assertions, and requires longer time to execute than other test cases. We use $APFD_{C-LOC}$, $APFD_{C-Assert}$, and $APFD_{C-Time}$ to denote the metric values of the three types of test cost, respectively.

5 Experimental Setup

This section presents the experimental design using 16 projects from Defects4J. We provide a preliminary analysis of the generated test suites in terms of their code coverage and mutation score, as well as test suite size and defect detection probability.

5.1 Test Projects and Defects

We use Defects4J 2.0.0 [50] to conduct our experiments. Defects4J is a database of more than 800 reproducible real defects mined from 17 Java projects. We use real defects because, to evaluate the effectiveness of mutation analysis, we require an external source of bugs. Each defect in Defects4J is isolated, with a buggy version and a fixed version. The difference between the buggy version and the fixed version only includes the defect. Although we perform mutation testing on the fixed version of each defect, we encounter failures in the mutation testing process. We briefly categorized the mutation failures occurring during three stages: compiling Mml (Major's domain-specific language) files (1.56%), mutating classes (17.12%), and running mutation analysis (81.32%). Details are in the Supplementary Material [2]. Therefore, we exclude the defects that fail to successfully complete mutation testing. We also exclude defects that could not produce mutation testing results within

Table 4. Information of Selected Defects in Defects4J 2.0.0

Project ID	Project Name	# Defects	# Mutants	# Tests
Chart	jfreechart	24	13,909	5,562
Cli	commons-cli	14	2,815	2,480
Closure	closure-compiler	69	25,858	31,096
Codec	commons-codec	18	13,056	1,783
Compress	commons-compress	41	19,969	5,683
Csv	commons-csv	13	2,348	1,774
Gson	gson	15	6,925	10,146
JacksonCore	jackson-core	26	50,250	6,413
JacksonDatabind	jackson-databind	62	13,919	74,710
JacksonXml	jackson-dataformat-xml	6	1,562	858
Jsoup	jsoup	77	29,557	25,272
JxPath	commons-jxpath	22	6,540	5,988
Lang	commons-lang	62	51,191	11,157
Math	commons-math	76	42,183	11,517
Mockito	mockito	19	593	11,167
Time	joda-time	21	9,747	48,516
Total		565	290,422	254,122

20 hours (i.e., potential hangs). This leaves 565 defects from 16 projects. The 565 retained defects are not a random subset of all defects. Since we only mutate classes that are modified due to a bug fix, the number of mutated classes is generally small. For example, the total time measured on our machine for performing mutation testing on 19 versions of the Mockito project is 29.9 hours. Therefore, mutation testing conducted in no more than 20 hours could include many defects. Table 4 summarizes the detailed information of the remained defects. Column *Project ID* and *Project Name* show the identifier and name of each project, respectively. Column # *Defects* shows the selected number of defects in each project.

5.2 Collection of Test Effectiveness Metrics

We use Cobertura [1] to obtain statement coverage and branch coverage of the code under test. Mutation testing is performed by Major [49]. We use all eight mutation operators in Major to run mutation analysis. The eight mutation operators are: Arithmetic Operator Replacement, Logical Operator Replacement, Shift Operator Replacement, Conditional Operator Replacement, Relational Operator Replacement, Literal Value Replacement, Operator Replacement Unary, and Statement Deletion. Table 4 presents the total number of generated mutants across all defects in each project in column # *Mutants*. The classes that are instrumented to obtain coverage information and are mutated are those modified by the bug fix, consistent with previous studies [97, 118]. Moreover, the code coverage collected and the mutation testing conducted are on the modified classes, which is the default setting in Defects4J. Code coverage and mutation score are measured and calculated on the fixed version of each defect. Since the test suites need to be aggregated over defects, we use normalized code coverage and mutation scores in this article. Code coverage is normalized by dividing the coverage achieved by all test cases in a test pool. The same method is applied to the mutation scores.

5.3 Method for Generating Test Suites

Previous studies have compared automatically generated test cases and manually written test cases [27, 28, 92]. These studies found that automatically generated test suites achieve higher structural coverage than manually written test suites, but do not lead to detection of more defects. In addition, as pointed out by Chen et al. [17], automatic test case generation tools generate subsequent test cases based on the generated test cases. Specifically, the Randoop tool [76], which is a feedback-directed random test case generation tool, uses previously valid test cases to build subsequent test cases. The

Algorithm 1: Generating Test Suites using Greedy Selection

Input: TEST_POOL, $covsc_{predefined}$
Output: TEST_SUITE

```
1:  test_suite ← {}
2:  covsc_prev ← 0.0
3:  while !IsEmpty(test_pool) do
4:      test_selected ← Random.Get(test_pool)
5:      covsc_cur ← Exec(test_suite ∪ {test_selected})
6:      if covsc_cur > covsc_prev then
7:          test_suite ← test_suite ∪ {test_selected}
8:          if covsc_cur >= covsc_predefined then
9:              break
10:     test_pool = test_pool − {test_selected}
```

EvoSuite tool [26] uses meta-heuristic search to generate test cases, which generate subsequent test cases by mutating generated test cases. Consequently, these test cases are dependent with each other. Therefore, similar to previous studies [6, 17, 45, 71], we use developer-written test cases in Defects4J to construct test suites.

All relevant tests in the developer-written tests form the *test pool* of the fixed version of each defect. Each test pool contains at least one triggering test case that can detect the defect. The size of the test pool for the 565 defects ranges from 1 to 4,011, with an average test pool size of 450 and a median of 230. Table 4, column *# Tests*, presents the total number of test cases across all defects in each project.

As shown in Table 1, many previous studies use the random selection approach to generate test suites [32, 45, 56, 58, 71, 77, 108]. However, there are studies that find that test suites constructed by random selection can cause threats to the conclusions. Zhang et al. [115] compare original test suites (the test suite comes with a project) and pseudo test suites (which are constructed by randomly selecting tests cases from original test suite). They find that pseudo test suites result in inflated correlation in measuring test effectiveness. Later, Chen et al. [17] discuss how the method of constructing a test suite by random selection is flawed. Therefore, we use *greedy selection*, a greedy algorithm that was used in previous studies [6, 96], to construct test suites.

According to Chen et al. [17], there are two variants of random selection: SizeFixed and SizeRandom. SizeFixed generates test suites of fixed size, for instance, in the range of 2.5–50% of the test pool size with 2.5% increments. SizeRandom constructs test suites of random size, for instance, in the range of 0–20% of the test pool size. Code coverage/mutation score are measured after test suites are generated using random selection. On the contrary, test suites constructed by greedy selection incrementally increase code coverage/mutation score without directly manipulating test suite size. Therefore, we generate test suites using the greedy selection algorithm, which is presented in Algorithm 1.

Algorithm 1 requires the test pool and the predefined code coverage/mutation score as input. The algorithm starts with an empty test suite (line 1). The initialization sets the previous code coverage/mutation score to zero, ensuring that generated test suites do not contain test cases whose code coverage/mutation score is zero (line 2). If the selected test case satisfies additional test goals, it is added to the test suite (lines 6–7). The selected test case is removed from the test pool (line 10). The process repeats until the test pool becomes empty (line 3) or the current code coverage/mutation score surpasses the predefined code coverage/mutation score (lines 8–9). The algorithm outputs the generated test suites.

For the fixed version of each defect, we generate different test suites that achieve different code coverage levels. We divide the coverage levels into 20 intervals, each interval length covering 5% of the coverage. Specifically, we iteratively generate code coverage of test suites greater than $c\%$ and less than or equal to $(c + 5)\%$, i.e., $(c\%, (c + 5)\%]$, where $c \in \{0, 5, 10, \ldots, 95\}$. The same procedure is applied to mutation score.

To construct test suites whose code coverage or mutation score falls within a specified interval, Algorithm 1 is modified to restart whenever the current code coverage/mutation score ($covsc_{cur}$) exceeds the predefined code coverage/mutation score ($covsc_{predefined} + 5$) by more than 5. In some cases, only one test suite's code coverage/mutation score falls within an interval $(c\%, (c + 5)\%]$. The constructed test suites are identical each time a test suite is constructed for the given interval. To avoid generating many identical test suites, we generate two test suites per interval. For the fixed version of each defect, a total of 40 test suites are generated. However, due to the presence of unreachable paths in the SUT, only a few test pools' code coverage/mutation scores (X_{max}) fall within the interval (95%, 100%]. Consequently, the maximum number of test suites generated for the fixed version of each defect is $40 \times c\% + 2$, where $X_{max} \in (c\%, (c + 5)\%]$. Additionally, code coverage/mutation score is not evenly distributed across the 20 intervals. When generating test suites with code coverage/mutation score falling within a given interval, if the number of failed attempts to generate test suites via Algorithm 1 exceeds the test pool size, we proceed to generate test suites whose code coverage/mutation score belongs to the next interval.

There are 254,122 test cases in total across the selected 565 defects, spanning all 16 projects. For each defect, we generated test suites using greedy selection based on different test effectiveness metrics. The test suites generated for each defect are aggregated across different defects under the same test effectiveness metric. Among the 565 defects, we generated a total of 54,876 test suites. Of these, 17,603, 18,044, and 19,229 test suites were generated for statement coverage, branch coverage, and mutation score, respectively.

5.4 Characteristics of Generated Test Suites

We conduct a preliminary analysis of the test suites constructed using the greedy selection approach. There are 3,613 and 4,225 test suites that detect defects in all test suites satisfying statement and branch coverage, respectively, which take up 20.52% and 23.41% of all test suites. For mutation score, there are 5,564 test suites (taking up 28.94%) that detect defects in the total 19,229 test suites. For all the constructed test suites, we calculate the probability of detecting defects at different intervals of code coverage or mutation score. Specifically, we calculate:

$$Pr\{Y = 1 | X \in (s\%, (s + 10)\%]\} = \frac{|\{X \in (s\%, (s + 10)\%)] \wedge (Y = 1)\}|}{|\{X \in (s\%, (s + 10)\%)]\}|}, \tag{18}$$

where X is the code coverage/mutation score of the constructed test suites, Y is whether the test suites detect a defect, and $s \in \{0, 10, 20, \ldots, 90\}$.

Figure 3 illustrates the probability of test suites detecting defects, distributed across 10 distinct intervals of code coverage or mutation score. In Figure 3, the test suites are aggregated from those generated for each defect. We analyzed 17,603 data points for test suites generated based on statement coverage, 18,044 for those based on branch coverage, and 19,229 for test suites constructed based on mutation score. We observe that the probability that a test suite detects a defect increases along with the increase in code coverage/mutation score. As shown in Figure 3, test suites that achieve statement coverage greater than 90%, branch coverage greater than 90%, or mutation score greater than 80% reach a probability of detecting defects greater than or equal to 0.5. Higher code coverage/mutation score has a higher probability of detecting defects. However, the probability of detecting defects is all below 0.71 for statement coverage, branch coverage, and mutation score in

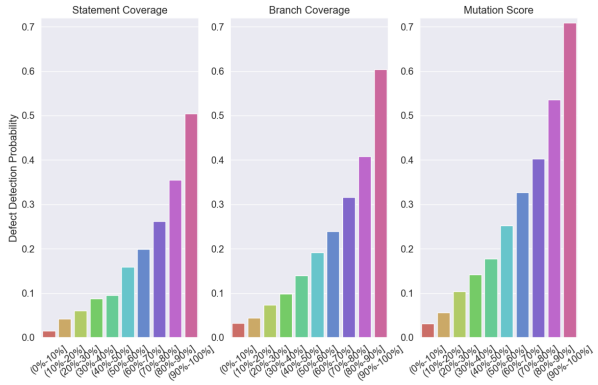


Fig. 3. The probability that test suites detect defects under different intervals of code coverage/mutation score.

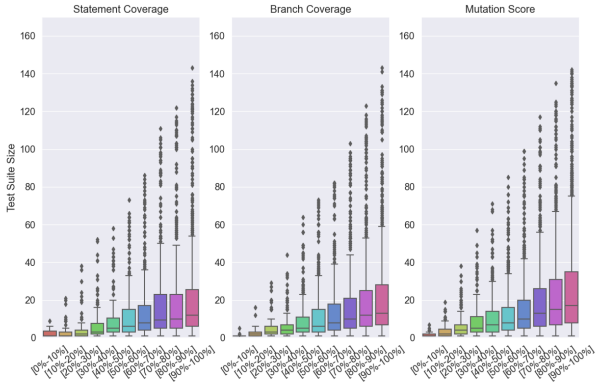


Fig. 4. Test suite size of test suites detecting defects under different intervals of code coverage/mutation score.

Figure 3. This observation suggests that only relying on statement coverage, branch coverage, or mutation score may not be sufficient to uncover all defects in the SUT.

For the test suites generated based on statement coverage, branch coverage, and mutation score, their sizes are 6.44%, 6.79%, and 8.01% of the test pool size on average across 565 defects, respectively, which is much smaller relative to the test pool size. Figure 4 displays the test suite size, i.e., $|TS_{X\in(s\%,(s+10)\%]\wedge(Y=1)}|$, across various intervals of code coverage or mutation score for test suites that detect defects. Obviously, the test suite size becomes larger along with the increase in code coverage/mutation score.

5.5 Measurement of Test Cost

In the evaluation of test case prioritization results, we consider three types of test costs for a test case: the number of lines of test code, the number of assertions, and the execution time. We measure the LOC in test case t by counting the number of non-comment and non-blank LOC in each test case, similar to the tool *cloc* [21]. There are different types of assertions in test cases, such as *assertTrue*(), *assertNotEquals*(), and *fail*(). The number of assertions in each test case is counted by traversing each test case. Since the time it takes to run a test case can vary each time it is executed,

Table 5. Total Effect of Test Effectiveness Metrics on Defect Detection

Project ID	Statement Coverage		Branch Coverage		Mutation Score	
	Estimate	p-value	Estimate	p-value	Estimate	p-value
Chart	0.043374	**<2e-16**	0.040936	**<2e-16**	0.044514	**<2e-16**
Cli	0.040072	**7.33e-11**	0.050293	**7.12e-15**	0.039380	**3.4e-14**
Closure	0.039510	**<2e-16**	0.05211	**<2e-16**	0.04896	**<2e-16**
Codec	0.032485	**1.45e-12**	0.035235	**<2e-16**	0.037181	**<2e-16**
Compress	0.046895	**<2e-16**	0.046457	**<2e-16**	0.050827	**<2e-16**
Csv	0.044553	**9.67e-10**	0.036720	**5.73e-10**	0.045455	**<2e-16**
Gson	0.046679	**1.93e-13**	0.049704	**<2e-16**	0.053602	**<2e-16**
JacksonCore	0.04416	**<2e-16**	0.04241	**<2e-16**	0.047402	**<2e-16**
JacksonDatabind	0.065525	**<2e-16**	0.060540	**<2e-16**	0.057211	**<2e-16**
JacksonXml	0.08197	**3.09e-08**	0.07880	**7.11e-8**	0.055469	**1.18e-10**
Jsoup	0.048865	**<2e-16**	0.045314	**<2e-16**	0.052919	**<2e-16**
JxPath	0.056443	**<2e-16**	0.044465	**<2e-16**	0.045142	**<2e-16**
Lang	0.033163	**<2e-16**	0.037122	**<2e-16**	0.036087	**<2e-16**
Math	0.037914	**<2e-16**	0.033380	**<2e-16**	0.03491	**<2e-16**
Mockito	0.06306	**1.35e-05**	0.048986	**9.83e-12**	0.046409	**6.98e-14**
Time	0.034431	**<2e-16**	0.039354	**<2e-16**	0.04651	**<2e-16**
Total	0.0401711	**<2e-16**	0.0414927	**<2e-16**	0.0436102	**<2e-16**

The p-values are adjusted using the Benjamini–Hochberg procedure.

we measure the execution time of each test case by executing each test case 10 times and taking the average of the 10 execution results.

5.6 Implementation

The experiments were mostly conducted on a server with an Intel(R) Xeon(R) Gold 6278C @ 2.60 GHz CPU, 16 GB memory, and Ubuntu 20.04 LTS operating system. The confounding effect analysis is conducted in R on macOS 13. We wrote Java and Python programs to implement our approach. The dataset and source code are available online [2].

6 Experimental Results

This section answers the six research questions using the results from the experiments. We further analyze our experimental results in comparison to previous studies.

6.1 RQ1: Estimation of Confounding Effect

We analyze the confounding effect using a large number of test suites rather than a single one. Instead of using data generated during the test suite construction process to test for the confounding effect, we add the data to the dataset only after a test suite is fully constructed for use in the confounding analysis. A total of 54,876 test suites were generated, resulting in 54,876 data points included in the analysis.

For each test effectiveness metric, we conduct data analysis on every project and perform an analysis that encompasses all projects. The confounding analysis is statistical rather than based on mathematical reasoning, making the analyses appropriately project-specific. The estimation of the confounding effect of test suite size, as well as the analysis of both the effect size and direction of the confounding effect, are based on the regression coefficients estimated for each project and across different projects.

We present the results of the confounding effect of test suite size on the association between test effectiveness metrics and defect detection. The *total effect* of test effectiveness metrics on defect detection is shown in Table 5. In Table 5, the column *Estimate* represents the estimates of the total effect, i.e., the regression coefficients of c in Equation (2). The column p-value presents the test results for the null hypothesis that the coefficient is 0. In Table 5, p-values below 0.05 are

Table 6. Indirect Effect of Test Effectiveness Metrics on Defect Detection

Project ID	Statement Coverage		Branch Coverage		Mutation Score	
	Estimate	Confidence Interval	Estimate	Confidence Interval	Estimate	Confidence Interval
Chart	0.0001	(−0.0036, 0.0040)	0.0001	(−0.0035, 0.0045)	0.0048	**(0.0015, 0.0087)**
Cli	−0.0014	(−0.0109, 0.0074)	−0.0039	(−0.0114, 0.0027)	0.0038	(−0.0024, 0.0109)
Closure	0.0098	**(0.0077, 0.0123)**	0.0056	**(0.0032, 0.0083)**	0.0062	**(0.0037, 0.0088)**
Codec	0.0174	**(0.0123, 0.0230)**	0.0110	**(0.0058, 0.0171)**	0.0059	**(0.0009, 0.0116)**
Compress	0.0067	**(0.0039, 0.0100)**	0.0057	**(0.0023, 0.0100)**	0.0099	**(0.0062, 0.0141)**
Csv	0.0016	(−0.0059, 0.0092)	−0.0016	(−0.0074, 0.0044)	0.0065	**(0.0007, 0.0137)**
Gson	−0.0016	(−0.0066, 0.0030)	0.0001	(−0.0044, 0.0048)	0.0007	(−0.0033, 0.0051)
JacksonCore	0.0064	**(0.0033, 0.0099)**	0.0033	**(0.0003, 0.0066)**	0.0010	(−0.0018, 0.0042)
JacksonDatabind	−0.0028	**(−0.0052, −0.0007)**	−0.0047	**(−0.0072, −0.0024)**	−0.0025	**(−0.0046, −0.0004)**
JacksonXml	0.0469	**(0.0280, 0.0760)**	0.0473	**(0.0294, 0.0787)**	0.0343	**(0.0176, 0.0598)**
Jsoup	0.0032	**(0.0013, 0.0052)**	0.0017	(−0.0002, 0.0036)	0.0012	(−0.0007, 0.0033)
JxPath	0.0067	**(0.0028, 0.0112)**	0.0067	**(0.0027, 0.0113)**	0.0098	**(0.0057, 0.0144)**
Lang	0.0043	**(0.0029, 0.0059)**	0.0056	**(0.0040, 0.0073)**	0.0051	**(0.0035, 0.0068)**
Math	0.0047	**(0.0033, 0.0062)**	0.0059	**(0.0045, 0.0076)**	0.0055	**(0.0040, 0.0071)**
Mockito	−0.0012	(−0.0065, 0.0019)	0.0047	(−0.0012, 0.0106)	−0.0001	(−0.0039, 0.0040)
Time	0.0201	**(0.0133, 0.0293)**	0.0330	**(0.0245, 0.0439)**	0.0203	**(0.0137, 0.0283)**
Total*	0.0049	**(0.0043, 0.0055)**	0.0052	**(0.0045, 0.0060)**	0.0046	**(0.0039, 0.0054)**
Total	0.0044	**(0.0039, 0.0050)**	0.0044	**(0.0038, 0.0050)**	0.0040	**(0.0034, 0.0046)**

shown in bold. The p-values are adjusted using the Benjamini–Hochberg procedure. We use the Benjamini–Hochberg procedure to control the false discovery rate when testing multiple hypotheses simultaneously. Since all p-values are less than 0.01, the total effect of statement coverage/branch coverage/mutation score on defect detection is significant. These results provide evidence that there is a statistically significant relationship between statement coverage/branch coverage/mutation score and defect detection. The row *Total* shows the results of the 16 projects. The results indicate that statement coverage, branch coverage, and mutation score are significantly related to defect detection across all projects.

We use the bootstrap method to test the *indirect effect*. Table 6 presents the results of the estimates of the *indirect effect* and the 95% bootstrap confidence interval for the *indirect effect* of statement coverage/branch coverage/mutation score on defect detection effectiveness. In Table 6, the 95% bootstrap confidence intervals that do not include zero are shown in bold. As can be seen, there are 11, 10, and 11 projects for statement coverage, branch coverage, and mutation score, respectively, that exhibit significant indirect effects. Row *Total** shows the results on the combination of the 11, 10, and 11 projects for statement coverage, branch coverage, and mutation score, respectively. Row *Total* presents the results across 16 projects. Although the significance test results indicate that several projects have insignificant indirect effects, the aggregate data from the test suites of all 16 projects suggest that test suite size does have a confounding effect.

The confounding effect of test suite size varies in different *projects*. For the three projects Cli, Gson, and Mockito, the 95% bootstrap confidence intervals for the indirect effect of statement coverage/branch coverage/mutation score on defect detection include zero, which indicates insignificant indirect effect. Besides, the confounding effect of test suite size varies across different *test effectiveness metrics*. For example, for the project Chart, zero is inside the 95% bootstrap confidence interval for both statement coverage and branch coverage, which indicates that the indirect effect is insignificant. However, the indirect effect of mutation score on defect detection effectiveness exhibits significant indirect effect.

We further analyze the 11, 10, and 11 projects that exhibit significant *indirect effect* of statement coverage/branch coverage/mutation score on defect detection. In Table 7, the columns *# Defects*, *# Mutants*, *# Tests*, and *# Test Suites* represent the number of defects, mutants, tests, and test suites, respectively. Rows 2–4 present the information for statement coverage, branch coverage, and mutation score across 11, 10, and 11 projects, respectively. Table 7 also presents detailed information

Table 7. Information on Projects Exhibiting Significant and Insignificant Indirect Effects

		# Defects	# Mutants	# Tests	# Test Suites
Total*	Statement Coverage	480	263,832	222,993	15,069
	Branch Coverage	403	234,275	197,721	12,881
	Mutation Score	414	200,282	198,644	14,149
JacksonXml	Statement Coverage	6	1,562	858	193
	Branch Coverage				208
	Mutation Score				223
Chart	Statement Coverage	24	13,909	5,562	766
Jsoup	Branch Coverage	77	29,557	25,272	2,556
	Mutation Score				2,728

Table 8. Direct Effect of Test Effectiveness Metrics on Defect Detection

Project ID	Statement Coverage		Branch Coverage		Mutation Score	
	Estimate	p-value	Estimate	p-value	Estimate	p-value
Chart	-	-	-	-	0.0403	**0.0000**
Closure	0.0265	**0.0000**	0.0447	**0.0000**	0.0422	**0.0000**
Codec	0.0128	**0.0185**	0.0226	**0.0000**	0.0303	**0.0000**
Compress	0.0382	**0.0000**	0.0395	**0.0000**	0.0401	**0.0000**
Csv	-	-	-	-	0.0386	**0.0000**
JacksonCore	0.0373	**0.0000**	0.0391	**0.0000**	-	-
JacksonDatabind	0.0690	**0.0000**	0.0662	**0.0000**	0.0600	**0.0000**
JacksonXml	0.0210	0.2576	0.0291	0.1202	0.0211	0.0679
Jsoup	0.0449	**0.0000**	-	-	-	-
JxPath	0.0494	**0.0000**	0.0373	**0.0000**	0.0358	**0.0000**
Lang	0.0293	**0.0000**	0.0328	**0.0000**	0.0320	**0.0000**
Math	0.0336	**0.0000**	0.0279	**0.0000**	0.0297	**0.0000**
Time	0.0144	**0.0044**	0.0073	0.2004	0.0266	**0.0000**
Total*	0.0353	**0.0000**	0.0360	**0.0000**	0.0378	**0.0000**
Total	0.0355	**0.0000**	0.0370	**0.0000**	0.0396	**0.0000**

The p-values are adjusted using the Benjamini–Hochberg procedure.

of three projects. For the project JacksonXml, the indirect effect of statement coverage/branch coverage/mutation score on defect detection is significant. JacksonXml has the *fewest* constructed test suites among projects with a significant indirect effect, totaling 193, 208, and 223 for statement coverage, branch coverage, and mutation score, respectively. In stark contrast, Chart has the *maximum* number of test suites constructed based on statement coverage among projects exhibiting an insignificant indirect effect. For test suites constructed based on branch coverage and mutation score, the maximum is found in the project Jsoup.

For the 11, 10, and 11 projects exhibiting significant indirect effects of statement coverage, branch coverage, and mutation score on defect detection, respectively, we test their *direct effects* on these projects. In Table 8, p-values below 0.05 are highlighted in bold. The *Total** row presents the aggregated results for these 11, 10, and 11 projects, respectively. The direct effects of code coverage and mutation score on defect detection are significant in different projects except for the project JacksonXml. Despite one project showing an insignificant direct effect, the overall direct effects of statement coverage, branch coverage, and mutation score on defect detection are significant when aggregating data from 11, 10, and 11 projects, respectively. While test suite size typically serves as a confounder, the presence of a significant direct effect implies that code coverage and mutation score continue to exert a substantial influence on defect detection capability.

Table 9 presents the *effect size* measures for the confounding effect. The estimate of the proportion confounded, calculated as $b/(b + ac')$, varies across different projects and test effectiveness metrics. The confounding effect and the direct effect of code coverage and mutation score on defect

Table 9. Effect Size for the Confounding Effect

Project ID	Statement Coverage	Branch Coverage	Mutation Score
Chart	-	-	0.1064
Closure	0.2700	0.1113	0.1281
Codec	0.5762	0.3274	0.1630
Compress	0.1492	0.1261	0.1980
Csv	-	-	0.1441
JacksonCore	0.1465	0.0778	-
JacksonDatabind	0.0390	0.0663	0.04
JacksonXml	0.6907	0.6191	0.6191
Jsoup	0.0665	-	-
JxPath	0.1194	0.1523	0.2149
Lang	0.1280	0.1458	0.1375
Math	0.1227	0.1746	0.1562
Time	0.5826	0.8189	0.4328
Total*	0.1219	0.1262	0.1085
Total	0.1103	0.1063	0.0917

detection exhibit opposite signs for the project JacksonDatabind. Following the method suggested by Alwin and Hauser [5], we take the absolute values of the indirect effects before computing the proportion confounded. This yields 3.9%, 6.63%, and 4.0% of the effects of statement coverage, branch coverage, and mutation score on defect detection, respectively, confounded by test suite size for the JacksonDatabind project. The confounding effect explains a minimum of 3.9% and a maximum of 69.07% of the total effect of *statement coverage* on defect detection. For *branch coverage*, the minimum and maximum proportions confounded are 6.63% and 81.89%, respectively. Additionally, a minimum of 4.0% and a maximum of 61.91% of the *mutation score*'s effect on defect detection is confounded by test suite size. In Table 9, the row *Total** shows the aggregate results across the 11, 10, and 11 projects that exhibit a significant indirect effect of statement coverage, branch coverage, and mutation score on defect detection, respectively. The proportions of the effects of statement coverage, branch coverage, and mutation score on defect detection that are confounded by test suite size are 12.19%, 12.62%, and 10.85%, respectively.

We judge the *direction* of the confounding effect for the 11, 10, and 11 projects that exhibit a significant indirect effect of statement coverage, branch coverage, and mutation score on defect detection, respectively. As shown in Tables 6 and 8, for the projects where test suite size acts as a confounder in the relationships between code coverage/mutation score and defect detection effectiveness, the signs of b/a are similar to c' in most projects. This indicates that the confounder has a positive effect [69], except for the project JacksonDatabind. Therefore, ignoring test suite size tends to inflate the crude relationship between test effectiveness metrics and defect detection in most projects. Conversely, in the relationship between test effectiveness metrics, test suite size, and defect detection, the sign of b/a differs from c' in the project JacksonDatabind. According to Mehio-Sibai et al. [69], the effect of ignoring test suite size is to deflate the crude relation, and the confounder has a negative effect. When combining the data from the 11, 10, and 11 projects, respectively, the sign of b/a is similar to c'. This indicates that test suite size has a positive effect and leads to an overestimation of the primary association between statement coverage, branch coverage, mutation score, and defect detection in general.

Chen et al. [17] suggest that using the random selection method to analyze the relationship between test effectiveness metrics, test suite size, and defect detection is flawed. Additionally, they claim that test suite size is not a confounding variable, based solely on test suites constructed via random selection and the presumed causal relation in the confounding model. In this article, we apply the confounding model to test suites generated without using random selection. Our statistical analysis of the confounding effect aligns with many previous studies [24, 67, 69, 119, 120],

Table 10. Test the Confounding Effect after Removing the Confounding Effect of Test Suite Size

Project ID	Statement Coverage		Branch Coverage		Mutation Score	
	Estimate	Confidence Interval	Estimate	Confidence Interval	Estimate	Confidence Interval
Chart	-	-	-	-	0.0000	(−0.0026, 0.0025)
Closure	0.0000	(−0.0014, 0.0013)	−0.0000	(−0.0018, 0.0017)	0.0000	(−0.0018, 0.0018)
Codec	−0.0000	(−0.0034, 0.0034)	0.0000	(−0.0035, 0.0036)	0.0000	(−0.0031, 0.0031)
Compress	−0.0000	(−0.0062, 0.0056)	0.0000	(−0.0059, 0.0061)	0.0000	(−0.0075, 0.0072)
Csv	-	-	-	-	0.0000	(−0.0049, 0.0054)
JacksonCore	−0.0000	(−0.0018, 0.0018)	−0.0000	(−0.0016, 0.0015)	-	-
JacksonDatabind	0.0000	(−0.0022, 0.0020)	0.0000	(−0.0023, 0.0023)	0.0000	(−0.0017, 0.0017)
JacksonXml	−0.0000	(−0.0219, 0.0192)	−0.0000	(−0.0302, 0.0262)	0.0000	(−0.0172, 0.0166)
Jsoup	0.0000	(−0.0020, 0.0016)	-	-	-	-
JxPath	−0.0000	(−0.0030, 0.0030)	−0.0000	(−0.0028, 0.0026)	0.0000	(−0.0033, 0.0029)
Lang	0.0000	(−0.0007, 0.0007)	0.0000	(−0.0008, 0.0008)	−0.0000	(−0.0007, 0.0007)
Math	−0.0000	(−0.0008, 0.0007)	0.0000	(−0.0008, 0.0008)	0.0000	(−0.0009, 0.0008)
Time	0.0000	(−0.0054, 0.0060)	0.0000	(−0.0104, 0.0108)	−0.0000	(−0.0049, 0.0052)
Total*	0.0000	(−0.0006, 0.0005)	−0.0000	(−0.0007, 0.0006)	−0.0000	(−0.0006, 0.0006)

which suggest an associative relationship between the independent variable and the third variable. The results indicate that the *indirect effect* is statistically significant, implying that test suite size impacts both test effectiveness metrics and defect detection capability. In previous studies [6, 39, 45, 77], test suite size is commonly regarded as having a confounding effect on the relationship between test effectiveness metrics and defect detection. The analysis result showing the existence of the confounding effect of test suite size is consistent with these previous studies.

> *Answer for RQ1*: In 11, 10, and 11 out of the 16 projects examined, the size of a test suite emerges as a confounding variable affecting the correlation between its test effectiveness metrics (statement coverage, branch coverage, and mutation score) and the capability of the test suite in defect detection. This influence of test suite size generally leads to an overestimation of their relationship, thereby amplifying their true association.

6.2 RQ2: Removal of Confounding Effect

We remove the confounding effect of test suite size in our analysis. Specifically, this pertains to the constructed test suites that demonstrate a significant indirect effect of statement coverage, branch coverage, and mutation score on defect detection across 11, 10, and 11 projects, respectively. This adjustment is accomplished through the application of linear regression, which is fitted based on the relationship between code coverage/mutation score and the test suite size. This regression is applied to the test suites constructed across the defects in each project under each test effectiveness metric. After removing the confounding effect of test suite size, we test the confounding effect using the bootstrap method. As shown in Table 10, zeroes are inside the 95% bootstrap confidence intervals for both code coverage and mutation score. This indicates that the indirect effect is insignificant and that the confounding effect has been successfully removed. By removing the confounding effect of test suite size, we obtain the clean code coverage and clean mutation score.

> *Answer for RQ2*: The deconfounding method based on linear regression effectively eliminates the impact of test suite size as a confounder in the relationship between test effectiveness metrics and defect detection.

6.3 RQ3: Evaluation of Test Effectiveness Metrics

We further investigate the real relationship between test effectiveness metrics and defect detection by utilizing the clean code coverage/mutation score. In the confounding analysis, the results indicate that test suite size is a positive confounder. The effect of test suite size leads to an overestimate of the crude relation between test effectiveness metrics and defect detection capability. Intuitively, the classification performance based on X_r is poorer than based on X regardless of the adopted models due to X_r not being influenced by test suite size.

Since some machine learning models are sensitive to the scale of the data [75], we perform feature scaling, also known as data normalization. Data normalization allows machine learning models to learn optimally and not skew toward larger scaled features [75]. Our chosen normalization method is z-score standardization, which recalibrates features to possess a mean of 0 and a standard deviation of 1. For implementing classification models, we leverage Scikit-learn [78], a prominent Python-based machine learning library. Through the functionalities offered by Scikit-learn, we construct our classification models.

According to the results of confounding analysis, 11, 10, and 11 projects containing 15,069, 12,881, and 14,149 test suites, respectively, exhibit significant indirect effects of statement coverage, branch coverage, and mutation score on defect detection effectiveness. The prediction is performed on three datasets composed of 15,069, 12,881, and 14,149 test suites, respectively. We train seven models (LR, DT, KNN, NB, MLP, SVM, and RF) for each dataset, with each dataset consisting of test suites constructed by statement coverage, branch coverage, and mutation score, respectively. For each dataset, we initially divided the data into two subsets: an 80% training set and a 20% test set. We then applied 5-fold cross-validation to the training set, dividing it into five equal parts, with the model being trained on four parts (64% of the entire dataset) and validated on the fifth part (16% of the entire dataset). This split is prevalent in studies that use machine learning for prediction tasks [22, 34, 48, 99]. The 5-fold cross-validation serves two purposes: it estimates the model's performance on unseen data to prevent overfitting and aids in optimizing the hyperparameters of the classification models. The optimal hyperparameters selected through GS for each model are available online [2].

Figure 5 presents the results of test suite effectiveness prediction. Figure 5(a)–(c) and (g)–(i) shows the average 5-fold cross-validation results on the validation set in terms of F1-score and G-mean, respectively. Figure 5(d)–(f) and (j)–(l) shows the results in the test set. Clearly, the clean code coverage/mutation score achieve lower predictive performance across all models compared to the original code coverage and original mutation score, in terms of both F1-score and G-mean. Since the clean code coverage/mutation score are free from the confounding effect of test suite size, the predictive performance indicates that test suite size explains the effectiveness of test suites, which are consistent with the results of confounding effect analysis.

We further investigate the degree to which test suite size impacts test suite effectiveness. We compare the improvements of predictive performance of different models which are constructed by the combination of code coverage/mutation score and test suite size. We use GS to find the optimal hyperparameters for each predictive model and then utilize each tuned predictive model to predict test suite effectiveness. Table 11 displays the results as the average metric values across both the validation and test sets. In Table 11, the maximum increase in predictive performance among the seven models is shown in bold. In terms of F1-score, the average predictive performance across seven models shows an increase of 3.92%, 1.13%, and 0.1% when test suite size is incorporated into classifications based on original statement coverage, branch coverage, and mutation score, respectively. Similarly, for classifications using clean statement coverage, branch coverage, and mutation score, the performance boosts are 9.88%, 9.48%, and 6.70% on average across the same

(a) Statement Coverage (b) Branch Coverage (c) Mutation Score

(d) Statement Coverage (e) Branch Coverage (f) Mutation Score

(g) Statement Coverage (h) Branch Coverage (i) Mutation Score

(j) Statement Coverage (k) Branch Coverage (l) Mutation Score

Fig. 5. Test suite effectiveness prediction using seven predictive models. The F1-score results are shown in subfigures (a)–(f), and the G-mean results are shown in subfigures (g)–(l).

seven models, respectively. However, the average predictive performance across the seven models, when using original mutation score and test suite size for classifications, is slightly lower than when based solely on the original mutation score. This is largely due to the significant decline in the predictive performance of NB when test suite size is incorporated with the original mutation score for classification. Overall, the results demonstrates that the combination of code coverage/mutation score and test suite size is superior to code coverage/mutation score in terms of predictive performance.

In Table 11, combining test suite size increases the maximum predictive performance by 14.79%, 14.21%, and 8.90% for clean statement coverage/clean branch coverage/clean mutation score in terms of F1-score, respectively. Assume that there are two test suites, TS_1 and TS_2, which possess identical code coverage/mutation score but differ in test suite size, where $|TS_1|$ is smaller than

Table 11. Improved Predictive Performance by Combining Test Suite Size with Code Coverage and Mutation Score

Predictive Model	Statement Coverage				Branch Coverage				Mutation Score			
	Original		Clean		Original		Clean		Original		Clean	
	F1-score	G-mean	F1-score	G-mean	F1-score	G-mean	F1-score	G-mean	F1-score	G-mean	F1-score	G-mean
Logistic Regression	5.10%	2.79%	11.75%	5.95%	0.18%	0.12%	**14.21%**	**9.54%**	0.54%	0.39%	8.31%	7.67%
Decision Tree	0.94%	1.21%	5.97%	4.85%	**3.39%**	**3.17%**	7.23%	6.37%	0.26%	0.69%	5.86%	5.57%
K-Nearest Neighbor	0.91%	−0.47%	4.95%	3.41%	2.10%	1.94%	8.00%	6.92%	0.39%	0.57%	6.16%	5.93%
Naïve Bayes	**14.95%**	**8.13%**	13.65%	7.17%	−3.44%	−2.85%	8.65%	5.08%	−2.99%	−3.75%	1.57%	1.36%
Multi-Layer Perceptron	2.68%	1.79%	**14.79%**	**8.91%**	1.17%	1.01%	9.76%	7.93%	**1.32%**	**1.42%**	8.78%	**8.40%**
Support Vector Machine	0.93%	0.70%	12.89%	7.36%	1.78%	1.66%	9.77%	8.12%	0.12%	−0.58%	**8.90%**	8.30%
Random Forest	1.89%	1.55%	5.15%	4.22%	2.70%	2.41%	8.72%	7.52%	1.08%	1.19%	7.28%	6.72%
Average	3.92%	2.24%	9.88%	5.98%	1.13%	1.06%	9.48%	7.35%	0.10%	−0.01%	6.70%	6.28%

Table 12. Test Suites for Bug 6 in the Project JxPath

ID	Test Suite
1	org.apache.commons.jxpath.ri.model.XMLSpaceTest[testPreserveDOM] **org.apache.commons.jxpath.ri.compiler.VariableTest**[testIterateVariable] org.apache.commons.jxpath.ri.model.XMLPreserveSpaceTest[testUnspecifiedJDOM] org.apache.commons.jxpath.ri.model.beans.BeanModelTest[testSetContextDependentNode]
2	org.apache.commons.jxpath.ri.model.dynabeans.DynaBeanModelTest[testBooleanPredicate] org.apache.commons.jxpath.ri.model.jdom.JDOMModelTest[testAxisFollowingSibling] org.apache.commons.jxpath.ri.model.dynamic.DynamicPropertiesModelTest[testCreatePathAndSetValue] org.apache.commons.jxpath.ri.model.XMLPreserveSpaceTest[testUnspecifiedDOM] org.apache.commons.jxpath.ri.model.dom.DOMModelTest[testNodeTypeText]

$|TS_2|$. In the scenario of selecting test suites for execution, the experimental results suggest opting for TS_2, as test suite size has an impact on defect detection effectiveness. However, the results do not necessarily mean that TS_1 must have a lower defect detection capability than TS_2. For instance, two test suites generated based on statement coverage of the bug 6 in the project JxPath are shown in Table 12. The test case that detects the defect is shown in bold in Table 12. The test suites both reach 61.5% statement coverage. The size of the test suites is 4 and 5, respectively. It is the test suite containing 4 test cases that detects the defect, rather than the large one.

In general, Figure 5 shows that mutation score is the most effective in predictive performance of different models in terms of F1-score and G-mean, while statement coverage is the least effective. We apply the Wilcoxon Signed-Rank test to analyze whether the differences in predictive performances between *clean* statement coverage, *clean* branch coverage, and *clean* mutation score are statistically significant. We compare the prediction results from both the validation set and the test set across seven models. The Wilcoxon Signed-Rank test is a non-parametric test and suitable for non-normal distributions. We also apply r effect size [29] for Wilcoxon Signed-Rank test to investigate the practical significance of differences. It is calculated as $r = z/\sqrt{N}$, where z is the z value for Wilcoxon Signed-Rank test, and N is the size of paired samples. We use Cohen's guidelines [19] for r: "negligible" for $|r| < 0.1$, "small" for $0.1 \leq |r| < 0.3$, "medium" for $0.3 \leq |r| < 0.5$, and "large" for $|r| \geq 0.5$. Table 13 presents the results of the comparison. The p-values below 0.05 are shown in bold. As shown in Table 13, all p-values are less than 0.05. The r effect size shows that the magnitude of their differences is large. Based on these results, we can conclude that there is a statistically significant difference in the predictive performances between statement coverage, branch coverage, and mutation score.

Our findings indicate that branch coverage more effectively predicts test suite effectiveness compared to statement coverage. Furthermore, using mutation score as a predictor for test suite effectiveness proves superior to relying on code coverage. Our results support the findings of previous studies [31, 32]. These studies compared different coverage criteria and found that branch

Table 13. Pairwise Comparison of the Predictive Performance of Statement Coverage, Branch Coverage, and Mutation Score

	F1-Score		G-Mean	
	p-value	Effect Size	p-value	Effect Size
SC vs. BC	**0.0002441**	−0.8641 (large)	**0.0001221**	−0.8808 (large)
SC vs. MS	**0.00018315**	−0.8808 (large)	**0.0001221**	−0.8808 (large)
BC vs. MS	**0.00018315**	−0.8808 (large)	**0.0001221**	−0.8808 (large)

SC, BC, and MS denote Statement Coverage, Branch Coverage, and Mutation Score, respectively.
The p-values, associated with Wilcoxon Signed-Rank test, are adjusted using the Benjamini–Hochberg procedure.

coverage is more effective than statement coverage in detecting defects. In addition, our results are consistent with several studies which conclude that mutation testing is better than code coverage in terms of defect detection [15, 51]. However, our results conflict with the findings of Gopinath et al. [35], which indicate that statement coverage is better than branch coverage in predicting mutation score, a proxy for actual defect detection. The contradiction between their conclusions and ours may arise from their finding that test suite size and cyclomatic complexity do not significantly contribute to mutation score. As a result, they directly analyze the correlation between code coverage and mutation score.

According to the results of confounding analysis, the direct relationship between test effectiveness metrics and defect detection (i.e., *direct effect*) is statistically significant. As shown in Figure 3, the probability of detecting defects remains consistently below 0.71 across statement coverage, branch coverage, and mutation score. In addition, the F1-score and G-mean of all seven models based on both code coverage/mutation score and test suite size are all blow 0.7. Furthermore, code coverage and the number of bugs detected are two commonly used evaluation metrics in fuzzy testing, evaluated in fuzzers including FairFuzz [60], NEUZZ [93], Cerebro [64], and SGF [80]. However, Böhme et al. [12] found that code coverage and the number of bugs detected show disagreement when used as evaluation metrics to assess fuzzy testing performance. Hence, beyond test suite size, other influencing factors might exist that impact the relationship between test effectiveness metrics and defect detection capability. These factors could potentially encompass the types of defects [79, 90] and program structure [14, 83].

Answer for RQ3: Test suite size helps explain defect detection effectiveness in predictive performance, aligning with identified confounding effects. After removing the confounding effect of test suite size, across various models, mutation score consistently emerges as the most reliable predictor of test suite effectiveness, as indicated by higher F1-scores and G-means, while statement coverage tends to be the least effective.

6.4 RQ4: Applicability to Alternative Measures of Test Suite Size

In the related work presented in Table 1, the 10 studies define test suite size based on the number of test cases. Additionally, we use the number of lines of test code and the number of assertions as alternative measures of test suite size.

The *total effect* is presented in Table 5 and remains consistent regardless of the test suite size measure adopted. Table 14 presents the *indirect effects* of test effectiveness metrics on defect detection, using the number of lines of test code as the measure for test suite size. Similarly, Table 15 illustrates the *indirect effects* when the number of assertions is used as the test suite size

Table 14. Indirect Effect of Test Effectiveness Metrics on Defect Detection Using the Number of Lines of Test Code as the Measure for Test Suite Size

Project ID	Statement Coverage		Branch Coverage		Mutation Score	
	Estimate	Confidence Interval	Estimate	Confidence Interval	Estimate	Confidence Interval
Chart	−0.0001	(−0.0040, 0.0041)	0.0009	(−0.0032, 0.0054)	0.0052	**(0.0018, 0.0090)**
Cli	−0.0015	(−0.0109, 0.0061)	−0.0044	(−0.0112, 0.0014)	0.0041	(−0.0016, 0.0107)
Closure	0.0085	**(0.0067, 0.0107)**	0.0060	**(0.0041, 0.0081)**	0.0066	**(0.0045, 0.0089)**
Codec	0.0103	**(0.0055, 0.0158)**	0.0063	**(0.0018, 0.0112)**	0.0014	(−0.0027, 0.0057)
Compress	0.0053	**(0.0025, 0.0082)**	0.0043	**(0.0012, 0.0080)**	0.0085	**(0.0051, 0.0124)**
Csv	0.0059	(−0.0031, 0.0143)	−0.0035	(−0.0110, 0.0038)	0.0062	**(0.0006, 0.0127)**
Gson	−0.0031	(−0.0082, 0.0015)	0.0002	(−0.0045, 0.0051)	0.002	(−0.0017, 0.0069)
JacksonCore	0.0075	**(0.0043, 0.0112)**	0.0031	**(0.0000, 0.0066)**	0.0009	(−0.0021, 0.0041)
JacksonDatabind	−0.0018	(−0.0042, 0.0002)	−0.0032	**(−0.0060, −0.0008)**	−0.0016	(−0.0037, 0.0005)
JacksonXml	0.0306	**(0.0149, 0.0542)**	0.0317	**(0.0179, 0.0533)**	0.0259	**(0.0118, 0.0453)**
Jsoup	0.0032	**(0.0013, 0.0051)**	0.0019	**(0.00001, 0.0038)**	0.0006	(−0.0012, 0.0027)
JxPath	0.0069	**(0.0030, 0.0114)**	0.0068	**(0.0025, 0.0118)**	0.0112	**(0.0068, 0.0161)**
Lang	0.0043	**(0.0028, 0.0059)**	0.0057	**(0.0041, 0.0075)**	0.0049	**(0.0033, 0.0068)**
Math	0.0051	**(0.0037, 0.0067)**	0.0055	**(0.0041, 0.0072)**	0.0049	**(0.0035, 0.0065)**
Mockito	−0.0013	(−0.0065, 0.0018)	0.0050	(−0.0007, 0.0109)	0.0007	(−0.0031, 0.0046)
Time	0.0189	**(0.0125, 0.0272)**	0.0297	**(0.0216, 0.0394)**	0.0198	**(0.0126, 0.0286)**
Total*	0.0055	**(0.0048, 0.0063)**	0.0051	**(0.0045, 0.0058)**	0.0056	**(0.0047, 0.0065)**
Total	0.0049	**(0.0043, 0.0054)**	0.0049	**(0.0043, 0.0056)**	0.0044	**(0.0038, 0.0051)**

Table 15. Indirect Effect of Test Effectiveness Metrics on Defect Detection Using the Number of Assertions as the Measure for Test Suite Size

Project ID	Statement Coverage		Branch Coverage		Mutation Score	
	Estimate	Confidence Interval	Estimate	Confidence Interval	Estimate	Confidence Interval
Chart	−0.0012	(−0.0052, 0.0027)	0.0001	(−0.0039, 0.0044)	0.0012	(−0.0023, 0.0048)
Cli	−0.0013	(−0.0092, 0.0059)	−0.0047	(−0.0113, 0.0010)	0.0023	(−0.0033, 0.0078)
Closure	0.0014	**(0.0003, 0.0027)**	−0.0004	(−0.0020, 0.0010)	0.0014	(−0.0002, 0.0031)
Codec	0.0016	(−0.0011, 0.0046)	0.0010	(−0.0015, 0.0037)	−0.0001	(−0.0022, 0.0021)
Compress	0.0056	**(0.0032, 0.0084)**	0.0039	**(0.0013, 0.0068)**	0.0076	**(0.0046, 0.0108)**
Csv	0.0076	(−0.0007, 0.0160)	−0.0069	(−0.0191, 0.0036)	0.0039	(−0.0027, 0.0106)
Gson	−0.0024	(−0.0071, 0.0020)	−0.0004	(−0.0047, 0.0043)	0.0027	(−0.0010, 0.0071)
JacksonCore	0.0074	**(0.0043, 0.0108)**	0.0031	**(0.0003, 0.0062)**	0.0006	(−0.0021, 0.0034)
JacksonDatabind	−0.0017	(−0.0039, 0.0003)	−0.0029	**(−0.0058, −0.0005)**	−0.0019	(−0.0041, 0.0002)
JacksonXml	0.0097	**(0.0001, 0.0232)**	0.0143	**(0.0052, 0.0268)**	0.0142	**(0.0045, 0.0267)**
Jsoup	0.0031	**(0.0013, 0.0050)**	0.0022	**(0.0004, 0.0041)**	0.0005	(−0.0013, 0.0025)
JxPath	0.0087	**(0.0044, 0.0136)**	0.0097	**(0.0050, 0.0149)**	0.0142	**(0.0092, 0.0199)**
Lang	0.0038	**(0.0025, 0.0052)**	0.0047	**(0.0033, 0.0064)**	0.0037	**(0.0024, 0.0053)**
Math	0.0045	**(0.0032, 0.0061)**	0.0045	**(0.0032, 0.0061)**	0.0039	**(0.0027, 0.0053)**
Mockito	−0.0013	(−0.0063, 0.0018)	0.0016	(−0.0030, 0.0059)	0.0025	(−0.0009, 0.0061)
Time	0.0163	**(0.0107, 0.0230)**	0.0234	**(0.0159, 0.0319)**	0.0130	**(0.0062, 0.0205)**
Total*	0.0032	**(0.0027, 0.0038)**	0.0036	**(0.0030, 0.0043)**	0.0033	**(0.0026, 0.0042)**
Total	0.0029	**(0.0024, 0.0034)**	0.0030	**(0.0025, 0.0035)**	0.0023	**(0.0018, 0.0028)**

measure. The *direct effects* of test effectiveness metrics on defect detection using the two alternative test suite size measures are presented in the Supplementary Material [2].

The presence of a confounding effect varies when using alternative measures for test suite size. Using the number of lines of test code as the test suite size measure, 10, 11, and 9 projects indicate that test suite size has a confounding effect on the relationship between statement coverage, branch coverage, mutation score, and defect detection, respectively. Similarly, when using the number of assertions as the test suite size, 9, 9, and 6 projects for statement coverage, branch coverage, and mutation score also show a confounding effect. Although results vary with different measures, the row *Total* indicates that the confounding effect exists across all generated test suites from all 16 projects, regardless of the test suite size measure used.

We remove the confounding effects of the number of lines of test code and the number of assertions by employing the linear regression-based method. Table 16 presents the test results

Table 16. Indirect Effect of Test Effectiveness Metrics on Defect Detection after Removing the Confounding Effect of the Number of Lines of Test Code

Project ID	Statement Coverage		Branch Coverage		Mutation Score	
	Estimate	Confidence Interval	Estimate	Confidence Interval	Estimate	Confidence Interval
Chart	-	-	-	-	0.0000	(−0.0027, 0.0025)
Closure	−0.0000	(−0.0012, 0.0011)	−0.0000	(−0.0016, 0.0015)	−0.0000	(−0.0016, 0.0016)
Codec	0.0000	(−0.0025, 0.0028)	0.0000	(−0.0024, 0.0026)	-	-
Compress	−0.0000	(−0.0047, 0.0041)	0.0000	(−0.0046, 0.0045)	0.0000	(−0.0062, 0.0058)
Csv	-	-	-	-	−0.0000	(−0.0048, 0.0046)
JacksonCore	−0.0000	(−0.0020, 0.0019)	0.0000	(−0.0017, 0.0016)	-	-
JacksonDatabind	-	-	−0.0000	(−0.0028, 0.0028)	-	-
JacksonXml	0.0000	(−0.0160, 0.0141)	−0.0000	(−0.0161, 0.0127)	−0.0000	(−0.0110, 0.0107)
Jsoup	−0.0000	(−0.0022, 0.0019)	0.0000	(−0.0022, 0.0020)	-	-
JXPath	0.0000	(−0.0030, 0.0031)	0.0000	(−0.0029, 0.0029)	0.0000	(−0.0035, 0.0033)
Lang	0.0000	(−0.0007, 0.0007)	−0.0000	(−0.0008, 0.0008)	0.0000	(−0.0007, 0.0007)
Math	0.0000	(−0.0010, 0.0009)	0.0000	(−0.0009, 0.0010)	0.0000	(−0.0010, 0.0009)
Time	0.0000	(−0.0051, 0.0058)	−0.0000	(−0.0091, 0.0096)	0.0000	(−0.0054, 0.0060)
Total*	0.0000	(−0.0006, 0.0006)	0.0000	(−0.0006, 0.0007)	−0.0000	(−0.0007, 0.0007)

Table 17. Indirect Effect of Test Effectiveness Metrics on Defect Detection after Removing the Confounding Effect of the Number of Assertions

Project ID	Statement Coverage		Branch Coverage		Mutation Score	
	Estimate	Confidence Interval	Estimate	Confidence Interval	Estimate	Confidence Interval
Closure	0.0000	(−0.0016, 0.0018)	-	-	-	-
Compress	−0.0000	(−0.0020, 0.0019)	0.0000	(−0.0026, 0.0024)	0.0000	(−0.0035, 0.0031)
JacksonCore	0.0000	(−0.0021, 0.0019)	−0.0000	(−0.0016, 0.0016)	-	-
JacksonDatabind	-	-	0.0000	(−0.0029, 0.0030)	-	-
JacksonXml	−0.0000	(−0.0115, 0.0108)	−0.0000	(−0.0124, 0.0102)	0.0000	(−0.0096, 0.0087)
Jsoup	−0.0000	(−0.0019, 0.0016)	0.0000	(−0.0019, 0.0018)	-	-
JXPath	0.0000	(−0.0036, 0.0036)	0.0000	(−0.0035, 0.0034)	−0.0000	(−0.0042, 0.0041)
Lang	0.0000	(−0.0006, 0.0006)	−0.0000	(−0.0007, 0.0007)	−0.0000	(−0.0006, 0.0006)
Math	0.0000	(−0.0008, 0.0009)	0.0000	(−0.0008, 0.0008)	0.0000	(−0.0008, 0.0008)
Time	0.0000	(−0.0042, 0.0045)	0.0000	(−0.0079, 0.0081)	0.0000	−0.0048, 0.0053)
Total*	−0.0000	(−0.0004, 0.0004)	−0.0000	(−0.0005, 0.0005)	−0.0000	(−0.0004, 0.0005)

after removing the confounding effect when using the number of lines of test code as the test suite size. Similarly, Table 17 shows the results when using the number of assertions as the test suite size. As shown in Tables 16 and 17, all of the 95% bootstrap confidence intervals include zero. Clearly, the confounding effects associated with these alternative test suite size measures have been successfully removed.

Following the same experimental setup described in Section 6.3, we use seven machine learning models to predict defect detection from code coverage and mutation score, free from the influence of the number of lines of test code, and the number of assertions, respectively. The optimal hyperparameters of the seven predictive models are presented in the Supplementary Material [2]. We apply the Wilcoxon Signed-Rank test and r effect size to analyze whether the differences in predictive performances between size-effect-free statement coverage, branch coverage, and mutation score are statistically significant, using the number of lines of test code as the test suite size measure. The same analysis is applied using the number of assertions as the test suite size measure. The results on the validation set are the average of the 5-fold cross-validation. Tables 18 and 19 present the pairwise comparison results.

In Tables 18 and 19, p-values less than 0.05 are highlighted in bold. As shown in Tables 18 and 19, all p-values are less than 0.05. The r effect size indicates that the magnitude of the differences in predictive performance is consistently large. Therefore, the mutation score has the highest predictive capability for defect detection, surpassing both statement coverage and branch coverage, which in turn shows greater predictive capability than statement coverage, regardless of the test

Table 18. Pairwise Comparison of the Predictive Performance of Statement Coverage, Branch Coverage, and Mutation Score after Removing the Confounding Effect of the Number of Lines of Test Code

	F1-Score		G-Mean	
	p-value	Effect Size	p-value	Effect Size
SC vs. BC	**0.0001221**	−0.8808 (large)	**0.0001221**	−0.8808 (large)
SC vs. MS	**0.0001221**	−0.8808 (large)	**0.0001221**	−0.8808 (large)
BC vs. MS	**0.0001221**	−0.8808 (large)	**0.0001221**	−0.8808 (large)

SC, BC, and MS denote Statement Coverage, Branch Coverage, and Mutation Score, respectively.
The p-values, associated with Wilcoxon Signed-Rank test, are adjusted using the Benjamini–Hochberg procedure.

Table 19. Pairwise Comparison of the Predictive Performance of Statement Coverage, Branch Coverage, and Mutation Score after Removing the Confounding Effect of the Number of Assertions

	F1-Score		G-Mean	
	p-value	Effect Size	p-value	Effect Size
SC vs. BC	**0.00018315**	−0.8808 (large)	**0.00018315**	−0.8808 (large)
SC vs. MS	**0.00018315**	−0.8808 (large)	**0.00018315**	−0.8808 (large)
BC vs. MS	**0.0003662**	−0.8473 (large)	**0.0002441**	−0.8641 (large)

SC, BC, and MS denote Statement Coverage, Branch Coverage, and Mutation Score, respectively.
The p-values, associated with Wilcoxon Signed-Rank test, are adjusted using the Benjamini–Hochberg procedure.

suite size measure used. These results align with those obtained when comparing test effectiveness metrics after removing the effect of test suite size, specifically using the number of test cases as the measure for test suite size.

> *Answer for RQ4*: The confounding effect of test suite size is present when using the number of lines of test code or the number of assertions as alternative test suite size measures. Regardless of which alternative measure is used, mutation score demonstrates the highest predictive capability for defect detection after successfully removing the confounding effect.

6.5 RQ5: Utility of Code Coverage in Predicting Mutation Score

The results presented in Section 6.3 indicate that the mutation score best predicts the effectiveness of the test suite. In this section, we assume that the mutation score serves as a proxy for defect detection and then evaluate whether statement coverage or branch coverage better predicts the mutation score.

The independent variable, X, is either statement coverage or branch coverage. Unlike Section 3, where the dependent variable, Y, is a dichotomous variable, Y is now a continuous variable. Therefore, we use linear regression instead of LR to explore the confounding effect of test suite size, Z.

The equation relating X and Y is shown in Figure 1(a):

$$Y = i_{Y^*} + cX + \varepsilon_{Y^*}, \tag{19}$$

where i_{Y^*} is the intercept, c is the regression coefficient, and ε_{Y^*} is the error term.

Table 20. Indirect Effect of Code Coverage on Mutation Score

Project ID	Statement Coverage		Branch Coverage	
	Estimate	Confidence Interval	Estimate	Confidence Interval
Chart	0.0273	**(0.0038, 0.0525)**	0.0315	**(0.0111, 0.0538)**
Cli	0.0436	**(0.0012, 0.0940)**	0.1074	**(0.0757, 0.1456)**
Closure	0.1488	**(0.1362, 0.1621)**	0.0837	**(0.0738, 0.0939)**
Codec	0.0436	**(0.0093, 0.0788)**	0.0483	**(0.0258, 0.0720)**
Compress	0.0478	**(0.0347, 0.0669)**	0.0247	**(0.0150, 0.0382)**
Csv	0.1479	**(0.1191, 0.1810)**	0.0480	**(0.0309, 0.0685)**
Gson	0.0525	**(0.0278, 0.0805)**	0.0212	**(0.0020, 0.0430)**
JacksonCore	0.1018	**(0.0865, 0.1184)**	0.0602	**(0.0490, 0.0724)**
JacksonDatabind	0.0487	**(0.0398, 0.0586)**	0.0208	**(0.0067, 0.0345)**
JacksonXml	0.0967	**(0.0653, 0.1331)**	0.0532	**(0.0275, 0.0797)**
Jsoup	0.0643	**(0.0543, 0.0750)**	0.0296	**(0.0214, 0.0382)**
JXPath	0.1084	**(0.0878, 0.1315)**	0.0677	**(0.0498, 0.0875)**
Lang	0.0317	**(0.0262, 0.0374)**	0.0193	**(0.0149, 0.0237)**
Math	0.0404	**(0.0334, 0.0480)**	0.0167	**(0.0111, 0.0226)**
Mockito	0.0273	**(0.0101, 0.0471)**	0.0623	**(0.0240, 0.1005)**
Time	0.0900	**(0.0758, 0.1052)**	−0.0005	(−0.0244, 0.0212)
Total*	0.0504	**(0.0480, 0.0528)**	0.0279	**(0.0258, 0.0299)**
Total	0.0504	**(0.0480, 0.0528)**	0.0276	**(0.0256, 0.0297)**

The confounder Z is related to both X and Y. The two equations corresponding to this relationship are shown in Figure 1(b):

$$X = i_X + aZ + \varepsilon_X, \tag{20}$$

$$Y = i_Y + c'X + bZ + \varepsilon_Y, \tag{21}$$

where i_X and i_Y are the intercepts, a, c', and b are the regression coefficients, ε_Z and ε_Y are the error terms.

Similar to the analysis in Section 3, the *total effect* of X on Y is represented by c. The *direct effect* of X on Y is denoted as c'. The *indirect effect* of X on Y, which is the bias due to the confounding, is the product of $\frac{1}{a}$ and b.

We use the percentile bootstrap method to test the confounding effect. In Table 20, we present the standardized indirect effect of X on Y. The total effect and the direct effect of X on Y are included in the Supplementary Material [2]. In Table 20, confidence intervals that do not include zero are shown in bold. As illustrated in Table 20, 16 projects exhibit a significant indirect effect of statement coverage on mutation score, and 15 projects show a similar effect for branch coverage.

Since test suite size has a confounding effect, we employ the linear regression-based method in Section 4.3 to remove this effect. Table 21 presents the test results obtained using the percentile bootstrap method after removing the confounding effect of test suite size. In Table 21, zeros fall within the 95% bootstrap confidence intervals, indicating that the confounding effect has been successfully removed.

In Section 6.3, we train seven models—LR, DT, KNN, NB, MLP, SVM, and RF—to predict defect detection using clean code coverage and clean mutation score, respectively. These seven models are used as classification models because defect detection is a dichotomous variable. In this section, the mutation score, a continuous variable, is used as a proxy for defect detection. Therefore, we adopt regression models to predict mutation score from clean statement coverage and clean branch coverage. Initially, we use linear regression since the confounding effect is modeled as linear. We then apply regression models corresponding to the six classification models—DT, KNN, NB, MLP, SVM, and RF—to predict mutation score from code coverage.

The dataset splitting is the same as in Section 6. We use GS to find the optimal hyperparameters for the seven regression models. The optimal hyperparameters of these models are presented in the

Table 21. Indirect Effect of Code Coverage on Mutation Score after Removing the Confounding Effect of Test Suite Size

Project ID	Statement Coverage		Branch Coverage	
	Estimate	Confidence Interval	Estimate	Confidence Interval
Chart	−0.0000	(−0.0455, 0.0436)	−0.0000	(−0.0488, 0.0455)
Cli	0.0000	(−0.0878, 0.0848)	−0.0000	(−0.0720, 0.0683)
Closure	0.0000	(−0.0233, 0.0230)	−0.0000	(−0.0263, 0.0249)
Codec	−0.0000	(−0.0413, 0.0411)	0.0000	(−0.0532, 0.0505)
Compress	−0.0000	(−0.0855, 0.0785)	0.0000	(−0.0860, 0.0806)
Csv	−0.0000	(−0.0954, 0.0861)	0.0000	(−0.0866, 0.0739)
Gson	−0.0000	(−0.0357, 0.0335)	−0.0000	(−0.0368, 0.0342)
JacksonCore	−0.0000	(−0.0305, 0.0298)	−0.0000	(−0.0294, 0.0278)
JacksonDatabind	0.0000	(−0.0320, 0.0311)	0.0000	(−0.0345, 0.0321)
JacksonXml	−0.0000	(−0.1363, 0.1215)	−0.0000	(−0.1654, 0.1414)
Jsoup	0.0000	(−0.0311, 0.0277)	0.0000	(−0.0324, 0.0297)
JXPath	−0.0000	(−0.0398, 0.0388)	−0.0000	(−0.0412, 0.0395)
Lang	0.0000	(−0.0117, 0.0115)	0.0000	(−0.0097, 0.0097)
Math	−0.0000	(−0.0116, 0.0110)	0.0000	(−0.0106, 0.0098)
Mockito	−0.0000	(−0.0268, 0.0286)	−0.0000	(−0.0476, 0.0435)
Time	0.0000	(−0.0657, 0.0643)	-	-
Total*	−0.0000	(−0.0082, 0.0080)	−0.0000	(−0.0084, 0.0078)

Table 22. Results of Predicting Mutation Score Using Code Coverage

Predictive Model	Clean Statement Coverage		Clean Branch Coverage	
	MSE	R^2	MSE	R^2
Linear Regression	0.0314	0.5920	0.0253	0.6761
Decision Tree Regression	0.0308	0.6002	0.0243	0.6880
K-Nearest Neighbors	0.0342	0.5560	0.0253	0.6754
Bayesian Regression	0.0314	0.5920	0.0253	0.6761
Multi-Layer Perceptron	0.0303	0.6062	0.0245	0.6859
Support Vector Regression	0.0303	0.6063	0.0233	0.7006
Random Forest Regression	0.0307	0.6016	0.0231	0.7035
Avg.	0.0313	0.5935	0.0245	0.6865

Supplementary Material [2]. We evaluate the regression models using MSE and R^2 Score, which are commonly used evaluation metrics. The prediction results of the seven regression models on the test set are presented in Table 22.

We compare the prediction results from the test set across seven regression models, calculated using two evaluation metrics. Since lower MSE results indicate better performance, we multiply each MSE result by −1 and then use the negative MSE results integrated with R^2 for comparison. We apply the Wilcoxon Signed-Rank test and calculate the r effect size to investigate whether there are statistically significant differences in predictive performance when using clean statement coverage and clean branch coverage to predict mutation scores. The p-value is 0.0001221, and the r effect size is −0.8808396, indicating that the magnitude of their difference is large. Therefore, branch coverage is a better predictor of mutation scores, which are strong indicators of defect detection.

> *Answer for RQ5*: The confounding effect of test suite size is present in the relationship between code coverage and mutation score. After removing the confounding effect, branch coverage appears to be a better predictor of mutation score than statement coverage.

6.6 RQ6: Influence on Test Suite Optimization

In this section, we present the outcomes of test suite reduction and test case prioritization based on both raw and adjusted code coverage/mutation score. The optimization process is executed on the

test pools corresponding to each defect in Defects4J. The original test pool encompasses test cases unable to cover any mutants. During test suite optimization using mutation score, we refine the test pool to a subset comprising tests from the original pool capable of covering mutants. When multiple test cases achieve identical code coverage or mutation score, we select one randomly during the optimization process. To mitigate uncertainty, we repeat the optimization algorithm five times for each defect's test pool.

6.6.1 Adjusted Test Effectiveness Metrics by Test Suite Size. The difference between the original and clean code coverage, as well as between the original and clean mutation score, is that the latter incorporates a *penalty* for test suite size. In this article, we employ a linear regression-based method to remove the confounding effect of test suite size. We obtain clean code coverage/mutation score X_r by subtracting the predicted values from original code coverage/mutation score X:

$$X_r = X - \hat{a_r} Z - \hat{i_X}. \tag{22}$$

To remove the confounding effect, we fit different equations (Equation (22)) using a large number of test suites for each project under different test effectiveness metrics. Inspired by the confounding removal method, for test suites generated within the same project under a single test effectiveness metric, we derive size-adjusted (approximately clean) test effectiveness metrics using the fitted equation. To distinguish between the original and approximately clean test effectiveness metrics, we refer to them as raw and adjusted test effectiveness metrics.

In the scenario of iteratively adding a test case to a test suite, if the code coverage/mutation score of a test suite has not yet reached its maximum value, the addition of a test case to the existing suite could potentially increase code coverage/mutation score. This expansion results in an increase of one in the test suite's size. Let X and X' represent the raw code coverage/mutation score in the current and previous iterations, respectively. Similarly, let X_r and X'_r denote the current and previous iterations' adjusted code coverage/mutation score. Across two consecutive iterations, if $X_r - X'_r > 0$, it indicates an enhancement in the adjusted code coverage/mutation score. The relationship can be expressed as:

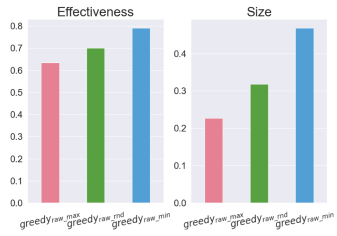
$$\begin{aligned} X_r - X'_r &= X - \hat{a_r} Z - \hat{i_X} - [X' - \hat{a_r}(Z-1) - \hat{i_X}] \\ &= X - X' - \hat{a_r}. \end{aligned} \tag{23}$$

The increase of the raw code coverage/mutation score simply requires X to be greater than X'. However, an increase in the adjusted code coverage/mutation score means that the raw code coverage/mutation score must increase by at least $\hat{a_r}$. Let x_{inc} represent the increase in the raw code coverage/mutation score between two consecutive iterations. If $x_{inc} > \hat{a_r}$, then the interval of $X > X'$ is $(0, x_{inc}]$, and the interval of $X_r > X'_r$ is $(\hat{a_r}, x_{inc}]$.

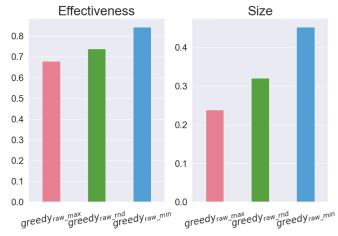
Once the raw code coverage/mutation score reaches its maximum value, adding more test cases to a test suite will no longer change this value. Conversely, according to Equation (22), when X remains constant but Z increases, X_r will continue to decrease. Therefore, the adjusted code coverage/mutation score of the test suite will decrease while the raw code coverage/mutation score remains unchanged.

6.6.2 Test Suite Reduction. Given that each defect in Defects4J is isolated, the FDL metric for reduced test suites has only two values: 0, indicating no defect detection loss, and 1, signifying that the reduced test suite fails to detect the defect.

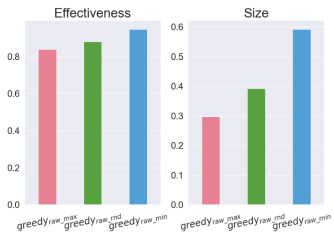
Test Suite Reduction Based on Raw Test Effectiveness Metrics. The greedy algorithm for test suite reduction iteratively selects test cases that have the maximum additional code coverage or mutation score. Chen et al. [17] discussed adequacy-based test suite reduction, which iteratively chooses a test case that has additional code coverage or mutation score. Since the reduced test suite has



(a) Statement Coverage



(b) Branch Coverage



(c) Mutation Score

Fig. 6. Test suite reduction using $greedy_{raw_max}$, $greedy_{raw_rnd}$, and $greedy_{raw_min}$.

the same code coverage or mutation score as the unreduced test suite, we further apply the approach that iteratively selects a test case that has the minimum additional code coverage or mutation score to perform test suite reduction. All three approaches require that the added test case to the reduced test suite increases the code coverage or mutation score, with the difference that the increments of code coverage or mutation score vary from maximum to minimum. We denote the three approaches as $greedy_{raw_max}$, $greedy_{raw_rnd}$, and $greedy_{raw_min}$, respectively. For $greedy_{raw_rnd}$, the increments of code coverage or mutation score at each iteration are in the interval $[greedy_{raw_min}, greedy_{raw_max}]$.

Figure 6 shows the results of the three approaches that reduce test suites based on raw statement coverage, raw branch coverage, and raw mutation score, respectively, across a total of 10, 9, and 10 projects. These projects exhibit a significant indirect effect of test suite size. We exclude the project JacksonDatabind from this analysis due to the high cost associated with conducting test suite optimizations. In Figure 6, the left subfigure represents the proportion of reduced test suites that detect defects out of the total reduced test suites, and the right subfigure signifies the ratio between the sum of the reduced test suites' sizes and the sum of the unreduced test suites' sizes. Apparently, as the increments of code coverage or mutation score decrease, i.e., from maximum to minimum, both the effectiveness and size of the reduced test suites increase, although the reduced test suites of the three approaches reach the same code coverage or mutation score. The results indicate that test suite size impacts defect detection effectiveness, which is consistent with the results of RQ1 and RQ3.

The reduction using $greedy_{raw_min}$ has the maximum defect detection effectiveness among the three approaches. However, on average, the size of the reduced test suites using $greedy_{raw_min}$ is the largest of the three methods. For example, for branch-based test suite reduction, the size of the reduced test suites using $greedy_{raw_min}$ is 1.8975 times and 1.4109 times larger than those using $greedy_{raw_max}$ and $greedy_{raw_rnd}$, respectively. To compare test suite reduction based on raw and adjusted code coverage/mutation score, we use $greedy_{raw_max}$, as it is the default greedy approach and is widely used [94].

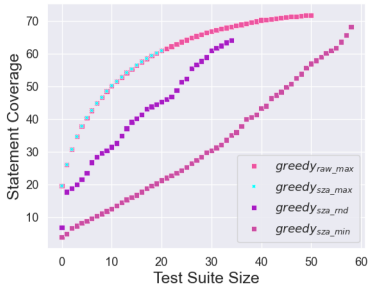


Fig. 7. Statement-based test suite reduction using $greedy_{raw_max}$, $greedy_{sza_max}$, $greedy_{sza_rnd}$, and $greedy_{sza_min}$ in bug 24 of project JacksonCore.

The PTSR and FDL results for raw statement coverage, raw branch coverage, and raw mutation score using $greedy_{raw_max}$ are an average of 85.94%, 85.10%, and 81.86%, and 36.65%, 32.08%, and 16.14%, respectively, across a total of 10, 9, and 10 projects. On the one hand, our results based on raw code coverage/mutation score are consistent with the study by Rothermel et al. [86, 87]. Rothermel et al. [86] found that the reduced test suites have 87–95% suite size reduction, while test suite reduction can reduce the defect detection capability of test suites by over 50%. They conclude that the defect detection capability of test suites can be severely compromised by minimization. On the other hand, our results based on raw code coverage/mutation score are in conflict with several previous studies [109, 110, 116], which concluded that test suite reduction can reduce test suite size without significantly reducing the defect detection capability. For instance, Wong et al. [109] found that test suite reduction has 1.19–44.23% suite size reduction while only experiencing 0–1.45% defect detection loss.

Test Suite Reduction Based on Adjusted Test Effectiveness Metrics. Similar to test suite reduction based on raw code coverage/mutation score, we define three approaches for performing test suite reduction based on adjusted code coverage/mutation score. These approaches involve: iteratively selecting a test case that has the maximum ($greedy_{sza_max}$), random ($greedy_{sza_rnd}$), and minimum ($greedy_{sza_min}$) additional adjusted code coverage/mutation score, respectively. Figure 7 presents the comparison results of statement-based reduction in bug 24 of project JacksonCore. The maximum coverage of all three approaches of adjusted statement-based reduction is lower than the raw statement-based reduction. As analyzed in Section 6.6.1, the adjusted code coverage/mutation score impose a penalty on test suite size; the adjusted code coverage/mutation score of the current iteration is greater than the previous iteration only if X_r is greater than $\hat{a}_r$, otherwise, the cumulative adjusted code coverage/mutation score will drop. If the reduction process stops once the cumulative adjusted code coverage/mutation score drops, the reduced test suites are *inadequate* for reduction based on adjusted code coverage and adjusted mutation score in most cases. This is because the corresponding cumulative raw code coverage/mutation score cannot reach the same level as the unreduced test suite. Here, inadequacy refers to the inability of a reduced test suite to achieve the same raw code coverage/mutation score as the unreduced test suite, rather than the inability to reach 100% raw code coverage/mutation score.

Since test suite reduction requires the reduced test suite satisfy the same test requirements as the unreduced test suite, we propose $greedy_{sza_adq}$—an approach based on adjusted code coverage/mutation score that satisfies the same test requirements as the unreduced test suite. Let $addtl_{cov_ms}(t_i) = X_r - X'_r$ represent the additional code coverage or mutation score for test case t_i between two consecutive iterations.

In the reduction process using $greedy_{sza_adq}$, whenever the adjusted code coverage/mutation score of the next iteration is less than or equal to that of the current iteration, i.e., $addtl_{cov_ms}(t_i) <= \hat{a_r}$, and not all test requirements are covered, we reset the cumulative adjusted code coverage/mutation score to zero and restart the process with the remaining test cases. After restarting, we arrange the remaining test cases $t_1, t_2, \cdots, t_n$ in decreasing order of additional adjusted code coverage/mutation score, such that $addtl_{cov_ms}(t_1) \geq addtl_{cov_ms}(t_2) \geq \cdots \geq addtl_{cov_ms}(t_n)$. For test cases where $addtl_{cov_ms}(t_k) > \hat{a_r}$, we begin with the test case having the maximum additional adjusted code coverage/mutation score and select a test case that can fulfill any uncovered test requirements. For example, in a scenario where test case t_1 fails to cover any uncovered test requirements, while test case t_2 succeeds in doing so, test case t_2 will be selected. If no test case t_k can satisfy any uncovered test requirements, the approach will opt for the first test case t_1. When the code coverage/mutation score of any remaining individual test case is less than or equal to $\hat{a_r}$, the reduction process becomes equivalent to $greedy_{raw_max}$. The process stops once the reduced test suites satisfy all test requirements.

Table 23 shows the results of test suite reduction using $greedy_{raw_max}$ and $greedy_{sza_adq}$. The average and median values for each project demonstrate that test suite reduction using $greedy_{raw_max}$ reduces test suite size more than using $greedy_{sza_adq}$, overall. On average, the PTSR is 80.51%, 79.87%, and 78.42% for reduction using $greedy_{sza_adq}$ based on statement coverage, branch coverage, and mutation score, respectively, across 10, 9, and 10 total projects. The corresponding average PTSR results using $greedy_{raw_max}$ are 85.94%, 85.10%, and 81.86%. The results demonstrate that test suite reduction using $greedy_{raw_max}$ achieves greater reductions in suite size compared to using $greedy_{sza_adq}$. As shown in the *Win/Tie/Loss* column in Table 23, in 10, 7, and 7 projects, reduction using $greedy_{raw_max}$ has higher defect detection loss compared to $greedy_{sza_adq}$ for reduction based on statement coverage, branch coverage, and mutation score, respectively. On average, the FDL is 29.47%, 26.10%, and 11.65% for reduction using $greedy_{sza_adq}$ based on statement coverage, branch coverage, and mutation score, respectively, across 10, 9, and 10 total projects. The results indicate that reduction using $greedy_{sza_adq}$ leads to less reduction in defect detection capability compared to using $greedy_{raw_max}$.

When comparing $greedy_{raw_max}$ and $greedy_{sza_adq}$, both the average PTSR and FDL values of $greedy_{raw_max}$ are higher than $greedy_{sza_adq}$ in general, although the higher FDL value indicates a larger defect detection loss. Our results further indicate that test suite reduction using $greedy_{sza_adq}$ has lower defect detection loss but comes with a larger test suite size. We use ΔPTSR to denote the average differences in PTSR between $greedy_{raw_max}$ and $greedy_{sza_adq}$, namely ΔPTSR = PTSR($greedy_{raw_max}$) − PTSR($greedy_{sza_adq}$). Similarly, we use ΔFDL to denote the average difference in FDL between the two approaches, i.e., ΔFDL = FDL($greedy_{raw_max}$) − FDL($greedy_{sza_adq}$). If ΔFDL/ΔPTSR is greater than 1, it means the increased defect detection effectiveness of $greedy_{sza_adq}$ is higher than the increased test suite size, which further indicates a better balance between the increased effectiveness and the size of reduced test suites. The results are shown in columns *ΔPTSR*, *ΔFDL*, and *ΔFDL/ΔPTSR* in Table 23.

As shown in Table 23, ΔFDL/ΔPTSR is greater than 1 in 6, 6, and 5 projects for statement-based, branch-based, and mutation-based test suite reduction, respectively. On average, using $greedy_{sza_adq}$ results in 5.43%, 5.23%, and 3.44% lower PTSR while having 7.18%, 5.98%, and 4.49% lower FDL compared to using $greedy_{raw_max}$ for reduction based on statement coverage, branch coverage, and mutation score across 10, 9, and 10 total projects, respectively. Overall, test suite reduction using $greedy_{sza_adq}$ balances test suite size and defect detection capability better than using $greedy_{raw_max}$.

Table 23. Comparison of Test Suite Reduction Using $greedy_{raw_max}$ and $greedy_{sza_adq}$

Project ID	PTSR				ΔPTSR	FDL			ΔFDL	ΔFDL/ΔPTSR	
	Avg.		Median			Win/Tie/Loss (%)	*Avg.*				
	Raw	Adjusted	Raw	Adjusted			Raw	Adjusted			
Closure	**0.9417**	0.9212	**0.9658**	0.9463	0.0205	2.32/86.96/**10.72**	0.4464	**0.3623**	0.0841	**4.1036**	Statement Coverage
Codec	**0.7576**	0.3844	**0.7590**	0.3333	0.3732	5.56/67.78/**26.67**	0.4333	**0.2222**	0.2111	0.5656	
Compress	**0.9234**	0.8769	**0.9508**	0.9213	0.0465	3.41/90.24/**6.34**	0.3415	**0.3122**	0.0293	0.6299	
JacksonCore	**0.7830**	0.6896	**0.9102**	0.7712	0.0934	0.0/94.62/**5.38**	0.2462	**0.1923**	0.0538	0.5766	
JacksonXml	**0.9284**	0.8543	**0.9219**	0.8522	0.0741	0.0/86.67/**13.33**	0.1333	**0.0**	0.1333	**1.7998**	
Jsoup	**0.9181**	0.8814	**0.9524**	0.9396	0.0367	0.0/94.55/**5.45**	0.4026	**0.3481**	0.0545	**1.4847**	
JxPath	**0.9635**	0.9507	**0.9700**	0.9696	0.0128	0.0/84.55/**15.45**	0.5000	**0.3455**	0.1545	**12.0606**	
Lang	**0.7052**	0.6536	**0.8086**	0.0516	0.0681	0.32/93.87/**5.81**	0.2290	**0.1742**	0.0548	**1.0623**	
Math	**0.8039**	0.7653	**0.8429**	0.8000	0.0387	3.95/88.42/**7.63**	0.3684	**0.3316**	0.0368	0.9529	
Time	**0.9571**	0.9320	**0.9893**	0.9766	0.0252	0.95/82.86/**16.19**	0.4381	**0.2857**	0.1524	**6.0543**	
Total*	**0.8594**	0.8051	**0.9284**	0.8913	0.0543	1.77/89.28/**8.95**	0.3665	**0.2947**	0.0718	**1.3223**	
Closure	**0.9251**	0.8963	**0.9483**	0.9314	0.0289	2.61/87.54/**9.86**	0.2754	**0.2029**	0.0725	**2.5099**	Branch Coverage
Codec	**0.7682**	0.4364	**0.7590**	0.2778	0.3318	0.0/73.33/**26.67**	0.4556	**0.1889**	0.2667	0.8037	
Compress	**0.9199**	0.8539	**0.9417**	0.8976	0.0661	0.0/93.17/**6.83**	0.3610	**0.2927**	0.0683	**1.0334**	
JacksonCore	**0.7627**	0.6909	**0.8989**	0.8241	0.0718	**4.62**/93.08/2.31	**0.2385**	0.2615	−0.0231	−0.3213	
JacksonXml	**0.9249**	0.8876	**0.9173**	0.8889	0.0373	0.0/100.0/0.0	0.1667	0.1667	0.0	0.0	
JxPath	**0.9624**	0.9498	0.9760	0.9760	0.0127	0.0/91.82/**8.18**	0.3455	**0.2636**	0.0818	**6.4608**	
Lang	**0.7336**	0.7146	**0.7931**	0.7457	0.0190	0.97/95.16/**3.87**	0.1968	**0.1677**	0.0290	**1.5254**	
Math	**0.8217**	0.7765	**0.8310**	0.8125	0.0452	1.84/91.32/**6.84**	0.4053	**0.3553**	0.0500	**1.1066**	
Time	**0.9670**	0.9588	0.9920	**0.9935**	0.0082	1.90/91.43/**6.67**	0.4571	**0.4095**	0.0476	**5.8008**	
Total*	**0.8510**	0.7987	**0.9167**	0.8795	0.0523	1.58/90.85/**7.57**	0.3208	**0.2610**	0.0598	**1.1441**	
Chart	**0.7944**	0.7738	**0.9136**	0.8337	0.0205	0.0/100.0/0.0	0.0833	0.0833	0.0	0.0	Mutation Score
Closure	**0.8987**	0.8625	**0.9234**	0.8741	0.0362	0.0/94.49/**5.51**	0.1072	**0.0522**	0.0551	**1.5217**	
Codec	**0.7355**	0.6475	**0.7343**	0.6024	0.0880	0.0/94.44/**5.56**	0.2778	**0.2222**	0.0556	0.6314	
Compress	**0.8855**	0.8503	**0.9080**	0.8706	0.0352	0.0/95.61/**4.39**	0.1415	**0.0976**	0.0439	**1.2485**	
Csv	**0.8715**	0.8161	**0.8750**	0.7949	0.0554	0.0/100.0/0.0	0.0769	0.0769	0.0	0.0	
JacksonXml	**0.9149**	0.8779	**0.9069**	0.8744	0.0369	0.0/100.0/0.0	0.1667	0.1667	0.0	0.0	
JxPath	**0.9544**	0.9383	**0.9618**	0.9600	0.0160	1.82/89.09/**9.09**	0.2545	**0.1818**	0.0727	**4.5316**	
Lang	**0.7000**	0.6745	**0.7641**	0.6932	0.0224	0.32/97.74/**1.94**	0.0839	**0.0677**	0.0161	0.6341	
Math	**0.7386**	0.7009	**0.8000**	0.7404	0.0377	0.79/92.11/**7.11**	0.2395	**0.1763**	0.0632	**1.6753**	
Time	**0.9613**	0.9436	**0.9856**	0.9663	0.0177	0.0/91.43/**8.57**	0.2667	**0.1810**	0.0857	**4.8495**	
Total*	**0.8186**	0.7842	**0.8817**	0.8405	0.0344	0.34/94.83/**4.83**	0.1614	**0.1165**	0.0449	**1.3032**	

The *Win/Tie/Loss* column indicates whether test suite reduction using $greedy_{raw_max}$ has a higher, equal, or lower defect detection effectiveness relative to reduction using $greedy_{sza_adq}$.
In the *Avg.*, *Median* and *Win/Tie/Loss* columns, better values are shown in bold.
In column Δ*FDL/*Δ*PTSR*, ΔFDL/ΔPTSR values greater than 1 are shown in bold.

> *Answer for RQ6.1*: Upon incorporating an adjustment for test suite size, both coverage-based and mutation-based test suite reduction methods demonstrate an improved balance between the reduction in test suite size and the incurred loss in defect detection.

6.6.3 Test Case Prioritization. As each defect is isolated in Defects4J, the simplified $APFD_C$ can be computed as:

$$APFD_C(simplified) = \frac{\sum_{i=TF_1}^{n} t_i - \frac{1}{2} \times t_{TF_1}}{\sum_{j=1}^{n} t_j}. \tag{24}$$

Test case prioritization aims to increase the speed of defect detection. The additional approach iteratively selects a test case with the maximum additional code coverage/mutation score. We denote this additional approach as $additional_{raw_max}$. Additionally, we represent the modified additional approach, which iteratively select a test case with the minimum additional code coverage/mutation score, as $additional_{raw_min}$. Intuitively, $additional_{raw_max}$ should perform better

than $additional_{raw_min}$ in terms of defect detection speed, and the experimental results supports this statement. For example, across 10 total projects, the average $APFD_{C-LOC}$, $APFD_{C-Assert}$, and $APFD_{C-Time}$ values of raw mutation score prioritization using $additional_{raw_min}$ are 0.0560, 0.0508, and 0.1067 lower than prioritization using $additional_{raw_max}$.

As analyzed in the test suite reduction process, when the adjusted code coverage/mutation score for the next iteration is less than or equal to that of the current iteration ($addtl_{cov_ms}(t_i) <= \hat{a}_r$) for the first time, it indicates that the currently partially prioritized test suites are generally inadequate. To ensure that uncovered statements and branches are executed, and uncovered mutants are killed as early as possible, we adopt an approach similar to $greedy_{sza_adq}$ in test case prioritization. We represent this approach as $additional_{sza_adq}$ and compare it with $additional_{raw_max}$.

Specifically, each time the cumulative adjusted code coverage/mutation score of the prioritized test suites cannot be increased, we reset the currently cumulative adjusted code coverage/mutation score, denoted as $local_{cov_ms}$, to zero. In the iterative process after resetting, we sort the remaining test cases based on additional adjusted code coverage/mutation score, such that $addtl_{cov_ms}(t_1) \geq addtl_{cov_ms}(t_2) \geq \cdots \geq addtl_{cov_ms}(t_n)$. For test cases where $addtl_{cov_ms}(t_k) > \hat{a}_r$, we select a test case that executes uncovered program elements or mutants. If no test case t_k can satisfy any uncovered test requirements, the approach defaults to selecting the first test case t_1. When the maximum code coverage or mutation score for each remaining test case is less than or equal to $\hat{a}_r$, the prioritization process becomes identical to $additional_{raw_max}$. We denote the total covered program elements or mutants in the SUT as $global_{cov_ms}$. If the $additional_{raw_adq}$ approach has executed all uncovered program elements or mutants and the prioritization process has not stopped, we reset $global_{cov_ms}$ to zero and continue using the additional approach to prioritize the remaining test cases.

Table 24 presents the comparison results. The *Avg.* column show the average $APFD_C$ values, with higher $APFD_C$ values in bold. The *Win/Tie/Loss* column indicates whether test case prioritization using $additional_{raw_max}$ has a higher, equal, or lower $APFD_C$ value compared to using $additional_{sza_adq}$, with better values shown in bold. We apply the Wilcoxon Signed-Rank test to investigate whether test case prioritization using $additional_{raw_max}$ and $additional_{sza_adq}$ is statistically significantly different. p-values below 0.05 are colored in pink.

As shown in Table 24, the average $APFD_C$ values of coverage-based test case prioritization using $additional_{sza_adq}$ are generally higher than those using $additional_{raw_max}$. In 7 and 3 projects, respectively, all three types of average $APFD_C$ values of statement-based and branch-based prioritization using $additional_{sza_adq}$ are higher than the average $APFD_C$ values using $additional_{raw_max}$. Across 10 total projects, the average $APFD_{C-LOC}$, $APFD_{C-Assert}$, and $APFD_{C-Time}$ values using $additional_{sza_adq}$ based on statement coverage are 0.8494, 0.8449, and 0.8667, respectively, while the average values based on branch coverage are 0.8356, 0.8329, and 0.8588, respectively, across a total of 9 projects, which are all higher than the corresponding values using $additional_{raw_max}$. In addition, the mutation-based test case prioritization results using $additional_{sza_adq}$ are 0.7779, 0.7634, and 0.8148 on average across 10 total projects, respectively, all of which are lower than those achieved using $additional_{raw_max}$.

The *Win/Tie/Loss* column compares the paired $APFD_C$ values obtained using $additional_{raw_max}$ and $additional_{sza_adq}$. Generally, the Win/Tie/Loss values indicate that statement-based and branch-based prioritization using $additional_{sza_adq}$ generates higher $APFD_C$ values, while the $APFD_{C-LOC}$, $APFD_{C-Assert}$, and $APFD_{C-Time}$ values using $additional_{sza_adq}$ for mutation-based prioritization only win in 2, 4, and 4 projects, respectively. However, the values in the *Win/Tie/Loss* column only show the number of times one $APFD_C$ value is higher, equal to, or lower than the other. It does not consider the magnitude of the differences. For example, in the Math project, mutation-based prioritization shows that $additional_{sza_adq}$ produces a higher average $APFD_{C-LOC}$ value compared

Table 24. Comparison of Test Case Prioritization Using $additional_{raw_max}$ and $additional_{sza_adq}$

Project ID	$APFD_{C-LOC}$ raw/adjusted		$APFD_{C-Assert}$ raw/adjusted		$APFD_{C-Time}$ raw/adjusted		
	Avg.	Win/Tie/Loss (%)	*Avg.*	Win/Tie/Loss (%)	*Avg.*	Win/Tie/Loss (%)	
Closure	0.9102/**0.9131**	**36.23**/28.12/35.65	0.9347/**0.9392**	22.90/46.96/**30.14**	0.9245/**0.9302**	35.36/19.71/**44.92**	Statement Coverage
Codec	0.7354/**0.7680**	25.56/23.33/**51.11**	0.7377/**0.7800**	27.78/27.78/**44.44**	0.7423/**0.7651**	30.0/13.33/**56.67**	
Compress	**0.8921**/0.8858	**43.4**/27.80/28.78	0.8500/**0.8519**	**34.15**/42.93/22.93	**0.8980**/0.8928	**47.32**/16.59/36.10	
JacksonCore	**0.8162**/0.8139	20.0/50.77/**29.23**	**0.8135**/0.8094	18.46/54.62/**26.92**	**0.8061**/0.8042	**37.69**/26.15/36.15	
JacksonXml	**0.9610**/0.9544	**36.67**/46.67/16.67	**0.9613**/0.9585	**33.33**/50.0/16.67	**0.9651**/0.9584	**43.33**/36.67/20.0	
Jsoup	0.8682/**0.8768**	32.47/29.35/**38.18**	0.8663/**0.8686**	32.21/30.65/**37.14**	0.8745/**0.8847**	**40.52**/19.22/40.26	
JxPath	0.9236/**0.9353**	17.27/39.09/**43.64**	0.9202/**0.9305**	19.09/40.0/**40.91**	0.9490/**0.9555**	30.91/26.36/**42.73**	
Lang	0.7607/**0.7661**	21.29/48.39/**30.32**	0.7589/**0.7648**	18.06/52.26/**29.68**	0.8143/**0.8211**	28.71/30.0/**41.29**	
Math	0.7800/**0.7826**	23.68/44.21/**32.11**	0.7578/**0.7651**	21.84/49.47/**28.68**	0.7976/**0.8011**	31.58/24.21/**44.21**	
Time	0.9477/**0.9499**	40.0/8.57/**51.43**	**0.9382**/0.9377	34.29/17.14/**48.57**	0.9555/**0.9571**	38.10/3.81/**58.10**	
Total*	0.8448/**0.8494**	29.47/35.31/**35.22**	0.8393/**0.8449**	25.26/42.63/**32.11**	0.8615/**0.8667**	35.74/21.58/**42.68**	
Closure	**0.9184**/0.9127	**37.39**/33.62/28.99	**0.9478**/0.9438	**31.01**/45.80/23.19	**0.9385**/0.9355	40.87/15.07/**44.06**	Branch Coverage
Codec	0.7434/**0.7872**	36.67/18.89/**44.44**	0.7560/**0.8129**	28.89/22.22/**48.89**	0.7527/**0.7815**	36.67/11.11/**52.22**	
Compress	**0.8800**/0.8791	**36.59**/29.27/34.15	0.8272/**0.8333**	27.32/40.0/**32.68**	0.8912/**0.8918**	**44.39**/12.68/42.93	
JacksonCore	**0.7955**/0.7861	**26.15**/48.46/25.38	**0.7913**/0.7845	**33.08**/40.0/26.92	**0.7887**/0.7815	30.77/33.85/**35.38**	
JacksonXml	0.9163/**0.9252**	23.33/50.0/**26.67**	0.9107/**0.9244**	23.33/50.0/**26.67**	0.9163/**0.9261**	26.67/43.33/**30.0**	
JxPath	**0.9407**/0.9359	**30.91**/40.0/29.09	**0.9350**/0.9309	30.0/39.09/**30.91**	**0.9585**/0.9574	**38.18**/25.45/36.36	
Lang	0.7705/**0.7714**	22.26/50.0/**27.74**	**0.7704**/0.7694	21.29/52.26/**26.45**	0.8329/**0.8339**	30.32/27.10/**42.58**	
Math	0.7524/**0.7622**	27.89/40.26/**31.84**	0.7351/**0.7453**	27.11/44.74/**28.16**	0.7723/**0.7836**	33.42/25.79/**40.79**	
Time	**0.9303**/0.9250	**38.10**/35.24/26.67	**0.9206**/0.9205	**37.14**/35.24/27.62	**0.9353**/0.9272	**51.43**/16.19/32.38	
Total*	0.8334/**0.8356**	**30.91**/38.71/30.38	0.8284/**0.8329**	28.15/43.34/**28.50**	0.8560/**0.8588**	36.95/21.82/**41.23**	
Chart	**0.7834**/0.7671	**25.0**/55.0/20.0	**0.7155**/0.7088	**22.5**/58.33/19.17	**0.8266**/0.8158	**30.0**/45.83/24.17	Mutation Score
Closure	**0.8992**/0.8929	**32.75**/41.74/25.51	**0.9201**/0.9137	**22.32**/59.13/18.55	**0.9241**/0.9193	**36.52**/31.59/31.88	
Codec	**0.7930**/0.7176	**44.44**/36.67/18.89	**0.8495**/0.7787	**35.56**/40.0/24.44	**0.8016**/0.7240	**51.11**/23.33/25.56	
Compress	**0.7829**/0.7594	**27.80**/50.24/21.95	0.7247/**0.7275**	21.46/54.15/**24.39**	**0.8140**/0.7928	**37.07**/35.61/27.32	
Csv	**0.8313**/0.7902	**40.0**/33.85/26.15	**0.8065**/0.7696	**40.0**/38.46/21.54	**0.8321**/0.8194	**49.23**/13.85/36.92	
JacksonXml	0.8011/**0.9076**	16.67/66.67/16.67	0.7850/**0.9060**	16.67/66.67/16.67	0.8108/**0.9134**	16.67/60.0/**23.33**	
JxPath	**0.7681**/0.7356	**33.64**/43.64/22.73	**0.7588**/0.7193	**32.73**/44.55/22.73	**0.8142**/0.7997	35.45/25.45/**39.09**	
Lang	0.6455/**0.6623**	16.45/58.06/**25.48**	0.6188/**0.6330**	14.52/60.65/**24.84**	0.7245/**0.7358**	28.06/38.71/**33.23**	
Math	0.7520/**0.7592**	**22.37**/57.89/19.74	0.7149/**0.7273**	18.42/62.37/**19.21**	0.7794/**0.7838**	**32.63**/38.42/28.95	
Time	0.9043/**0.9091**	19.05/48.57/**32.38**	0.9043/**0.9060**	20.0/50.48/**29.52**	0.9118/**0.9213**	33.33/20.0/**46.67**	
Total*	**0.7838**/0.7779	**26.36**/50.40/23.24	**0.7649**/0.7634	21.76/56.42/**21.82**	**0.8190**/0.8148	**34.43**/34.09/31.48	

to $additional_{raw_max}$. However, the Win/Tie/Loss analysis suggests that $additional_{raw_max}$ yields a greater number of instances with higher $APFD_{C-LOC}$ values than $additional_{sza_adq}$.

Statistically significant differences are observed in statement-based, branch-based, and mutation-based prioritization for 7, 3, and 6 projects, respectively, when comparing prioritization using $additional_{raw_max}$ and $additional_{sza_adq}$. The statistically significant difference varies depending on the *test effectiveness metric*. For example, in the JxPath project, the $APFD_C$ values of branch-based prioritization are not significantly different, while the p-values of statement-based prioritization indicate significant differences. The statistically significant difference in the test case prioritization results varies between *types of test costs*. For instance, for mutation-based prioritization employing $additional_{raw_max}$ and $additional_{sza_adq}$ in project Lang, the p-values for $APFD_{C-LOC}$ and $APFD_{C-Assert}$ indicate significant differences, whereas the p-value for $APFD_{C-Time}$ does not. Overall, the results of the Wilcoxon Signed-Rank test suggest small disparities in the prioritization outcomes of $additional_{raw_max}$ and $additional_{sza_adq}$.

Answer for RQ6.2: Upon incorporating an adjustment for test suite size, there is a marginal improvement in the speed of defect detection for both statement coverage-based and branch coverage-based test case prioritization.

7 Discussion

We discuss the characteristics of test suites constructed by random selection and greedy selection, as well as how the test suites constructed through greedy selection satisfy the three criteria for identifying a confounder. To substantiate the significance of the confounding effect, we utilize two additional statistical test approaches for further scrutiny. Next, we discuss whether the original or the clean test effectiveness metrics are being used. Furthermore, we discuss the use of **Probabilistic Coupling (PC)** in comparing test effectiveness metrics and highlight the differences between PC and our work. This section also delves into the implications of our experimental outcomes and examines potential threats to the validity of the study.

7.1 Characteristics of Test Suites Constructed by Random Selection and Greedy Selection

There are two main approaches for constructing test suites in studies analyzing the relationship between code coverage or mutation score and defect detection capabilities: random selection [32, 45, 56, 58, 71, 77, 108] and greedy selection [6, 17, 96].

Random Selection. In random selection, test suites are constructed based on predefined sizes. For example, Inozemtseva and Holmes [45] constructed a total of 31,000 test suites through random selection. They constructed 1,000 test suites with sizes of 3 test cases, 10 test cases, 30 test cases, 100 test cases, and so on. However, random selection is far from a practical way to construct test suites. Chen et al. [17] argue that test suite size is an unrealistic test objective.

Random selection is less realistic than greedy selection due to the subjectively determined test suite size. Constructing test suites based solely on their size is not practical. Some studies [61, 62] augment test suites to 200 or more test cases for testing synthesized programs, but this number is not universally applicable as a guideline for software testing.

Greedy Selection. For greedy selection, the test suite generation process stops once the generated test suite achieves a code coverage or mutation score surpassing a predefined threshold. The test suites were constructed based on code coverage or mutation score, rather than by directly specifying test suite size, as in random selection. The size of the generated test suites is not fixed but varies and is measured after the generation process is complete. In our study, we adopt greedy selection due to the limitations of random selection in constructing test suites. This approach is used to construct a large number of test suites.

In studying the relationship between code coverage or mutation score and defect detection, there are previous studies [6, 17, 96] that use greedy selection to construct test suites. Although we generate test suites using greedy selection, our analysis results show that test suite size is a confounder. These results are consistent with previous studies [45, 77] that used random selection to construct test suites and controlled for the confounding effect of test suite size.

The greedy selection approach resembles the process of covering uncovered parts of the program under test or killing survived mutants. According to the PIE theory [106], test suites cannot detect defects in parts of the program they never execute. Mutants can be regarded as a proxy for defect detection, and killing more mutants may improve the defect detection capability of test suites. This process is presented in many software engineering scenarios, such as greybox fuzzy testing [10, 11], search-based testing [44], constraint-based testing [13, 91, 103], and test case generation using large language models [20, 36]. Rojas et al. [85] present a gamified and crowdsourced approach to mutation testing where players create tests and mutants through competitive gameplay, resulting in higher quality test suites.

7.2 Satisfaction of the Criteria for Identifying a Confounder in Test Suites Constructed through Greedy Selection

In this section, we specifically discuss how the test suites constructed through greedy selection satisfy the three criteria for identifying a confounder.

7.2.1 Criterion (1): It Should Be Related to the Independent Variable. For test suites that are generated through greedy selection and have finished construction, a large test suite size tends to have high code coverage or mutation score; high code coverage or mutation score also tends to be associated with a large test suite size. Therefore, criterion (1) for identifying a confounder is fulfilled.

7.2.2 Criterion (2): It Should Be a Risk Factor for or a Predictor of the Dependent Variable. In the process of constructing a test suite in our study, since we iteratively select a test case that enhances code coverage or mutation score, the size of the test suite increases as code coverage or mutation score improves. Test suite size depends on code coverage or mutation score only within the constraints of a process that constructs a single test suite.

However, the confounding effect is analyzed using a large number of test suites rather than a single test suite. In our study, we construct test suites based on code coverage or mutation score to analyze the relationship between code coverage/mutation score and defect detection. After generating test suites based on code coverage or mutation score, we measure their sizes and assess whether they can detect defects. For different test suites, high code coverage or mutation score does not necessarily lead to a large test suite size.

For example, the statement coverage, test suite size, and defect detection data for two test suites generated in Mockito_16 are as follows: $|T_1| = 79.0$, 23, 0, and $|T_2| = 86.0$, 19, 0. Although T_2 has higher statement coverage, its size is smaller than T_1. Furthermore, neither T_1 nor T_2 detects defects. For the test suites constructed in our study, even if the test suites have the same code coverage or mutation score, their sizes may differ. We provide an example in Section 6.3. Table 12 presents two test suites generated based on the statement coverage of bug 6 in the JxPath project. Both test suites achieve 61.5% statement coverage, but their sizes are 4 and 5, respectively.

Van Stralen et al. [105] present an example in which obesity (Z) is considered to be a confounder in the relationship between hypertension (X) and ESRD (Y). For an individual, obesity is associated with hypertension; they are not two independent processes and are not independent of each other. The confounding analysis is conducted based on data from many individuals. For a large population, obesity is associated with hypertension, as many obese patients develop hypertension.

Thus, for test suites constructed through greedy selection, test suite size is conceptually independent of code coverage or mutation score.

Previous studies [6, 45, 58, 71, 77, 108] have shown that test suite size influences defect detection. These studies use random selection and greedy selection to construct test suites, and the influence of test suite size on defect detection is independent of the test suite construction process (not tied to a particular data-generating process). Moreover, in test suite reduction, while code coverage or mutation score remains unchanged, reducing the test suite size to eliminate redundant test cases can lead to a decrease in defect detection capability, although the impact may vary in significance.

For the example presented by Van Stralen et al. [105], obesity is a factor that is related to ESRD as many obese patients develop ESRD.

Thus, test suite size is a predictor, a surrogate for a causal factor, and could be a determinant of defect detection (criterion 2 fulfilled).

7.2.3 Criterion (3): It Should Not Be Part of the Causal Pathway from the Independent Variable to the Dependent Variable. The confounding effect is analyzed using a large number of test suites

rather than a single test suite. Specifically, for test suites generated by greedy selection based on the same programs under test, high code coverage or mutation score does not necessarily imply a large test suite size, which in turn leads to high defect detection capability.

Take, for example, the test suites generated under statement coverage in Lang_16. The statement coverage, test suite size, and defect detection data for two test suites are as follows: $|T_1| = 51.6$, 40, 0, and $|T_2| = 54.2$, 16, 1. Although T_2 has higher statement coverage, its size is smaller than T_1. It is the smaller-sized test suite T_2 that detects defects, rather than T_1. Similarly, for the test suites generated in Lang_16, another two test suites generated based on statement coverage are $|T_3| = 82.6$, 44, 0 and $|T_4| = 83.4$, 41, 1. Moreover, even if the test suites have the same code coverage or mutation score and are based on the same programs under test, their sizes are not the same, nor is their defect detection capability. We have provided an example in Table 12 in Section 6.3.

There are many such examples among the 54,876 test suites generated using greedy selection in our study. For different test suites generated using the greedy selection approach, there is no inherent property that high code coverage or mutation score must lead to a large test suite size and, consequently, high defect detection capability.

Therefore, the test suites constructed through greedy selection align with the third criterion for identifying a confounder.

7.3 Test the Confounding Effect by Other Approaches

In Table 6, we presented the results of testing the confounding effect using percentile bootstrap. There are other approaches to test confounding effects. A previous study [41] found that both the bias-corrected bootstrap and the Monte Carlo [72, 113] are more powerful and more trustworthy than the Sobel test. The bias-corrected bootstrap is more powerful but has less accurate Type I error rates than Monte Carlo [41]. In this section, we use bias-corrected bootstrap and Monte Carlo to test the confounding effect of test suite size.

The bias-corrected bootstrap has a higher statistical power compared with other tests [41] but has excess Type I error rates in several cases [30]. We present the test results using bias-corrected bootstrap. Similar to percentile bootstrap, we request 5,000 bootstrap samples. Table 25 shows the indirect effect of code coverage/mutation score on defect detection effectiveness. Column *Confidence Interval* shows the lower and upper confidence limits of the 95% bias-corrected bootstrap confidence interval. In Table 25, the 95% bias-corrected bootstrap confidence interval that does not include zero is shown in bold. For 11, 10, and 11 projects, there are significant *indirect effects* of statement coverage, branch coverage, and mutation score on defect detection, respectively. The results are consistent with the test results using the percentile bootstrap method.

In addition to utilizing the bias-corrected bootstrap, we also apply the Monte Carlo method to test the confounding effect of test suite size on the relationship between test effectiveness metrics and defect detection. We set the number of Monte Carlo samples to 5,000. The estimates of the indirect effect and the 95% Monte Carlo confidence intervals are presented in Table 26. The 95% Monte Carlo confidence intervals that do not include zero are shown in bold in Table 26. For 11, 10, and 11 projects, the results exhibit significant *indirect effects* of statement coverage, branch coverage, and mutation score on defect detection effectiveness, respectively. The results of the Monte Carlo are consistent with the test results using the percentile bootstrap method.

The two methods, i.e., the bias-corrected bootstrap and the Monte Carlo, further confirm that test suite size has a confounding effect on the relationship between test effectiveness metrics and defect detection capability.

Table 25. Test the Confounding Effect Using the Bias-Corrected Bootstrap Method

Project ID	Statement Coverage		Branch Coverage		Mutation Score	
	Estimate	Confidence Interval	Estimate	Confidence Interval	Estimate	Confidence Interval
Chart	0.0001	(−0.0036, 0.0040)	0.0001	(−0.0036, 0.0043)	0.0048	**(0.0015, 0.0086)**
Cli	−0.0014	(−0.0109, 0.0074)	−0.0039	(−0.0112, 0.0028)	0.0038	(−0.0023, 0.0110)
Closure	0.0098	**(0.0077, 0.0123)**	0.0056	**(0.0032, 0.0081)**	0.0062	**(0.0036, 0.0087)**
Codec	0.0174	**(0.0122, 0.0228)**	0.0110	**(0.0056, 0.0169)**	0.0059	**(0.0008, 0.0116)**
Compress	0.0067	**(0.0038, 0.0099)**	0.0057	**(0.0022, 0.0098)**	0.0099	**(0.0062, 0.0141)**
Csv	0.0016	(−0.0060, 0.0091)	−0.0016	(−0.0074, 0.0043)	0.0065	**(0.0005, 0.0134)**
Gson	−0.0016	(−0.0066, 0.0029)	0.0001	(−0.0045, 0.0048)	0.0007	(−0.0034, 0.0049)
JacksonCore	0.0064	**(0.0033, 0.0099)**	0.0033	**(0.0003, 0.0066)**	0.0010	(−0.0019, 0.0040)
JacksonDatabind	−0.0028	**(−0.0051, −0.0006)**	−0.0047	**(−0.0071, −0.0023)**	−0.0025	**(−0.0045, −0.0004)**
JacksonXml	0.0469	**(0.0259, 0.0724)**	0.0473	**(0.0269, 0.0730)**	0.0343	**(0.0165, 0.0573)**
Jsoup	0.0032	**(0.0013, 0.0052)**	0.0017	(−0.0002, 0.0036)	0.0012	(−0.0007, 0.0033)
JxPath	0.0067	**(0.0028, 0.0028)**	0.0067	**(0.0024, 0.0109)**	0.0098	**(0.0057, 0.0144)**
Lang	0.0043	**(0.0029, 0.0059)**	0.0056	**(0.0040, 0.0073)**	0.0051	**(0.0035, 0.0068)**
Math	0.0047	**(0.0033, 0.0062)**	0.0059	**(0.0044, 0.0076)**	0.0055	**(0.0039, 0.0070)**
Mockito	−0.0012	(−0.0063, 0.0019)	0.0047	(−0.0012, 0.0105)	−0.0001	(−0.0040, 0.0038)
Time	0.0201	**(0.0128, 0.0285)**	0.0330	**(0.0244, 0.0436)**	0.0203	**(0.0137, 0.0283)**
Total*	0.0049	**(0.0043, 0.0055)**	0.0052	**(0.0045, 0.0060)**	0.0046	**(0.0038, 0.0054)**
Total	0.0044	**(0.0039, 0.0050)**	0.0044	**(0.0038, 0.0050)**	0.0040	**(0.0034, 0.0046)**

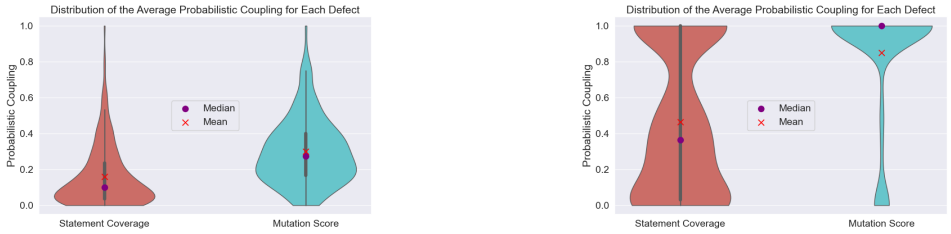
Table 26. Test the Confounding Effect Using the Monte Carlo Method

Project ID	Statement Coverage		Branch Coverage		Mutation Score	
	Estimate	Confidence Interval	Estimate	Confidence Interval	Estimate	Confidence Interval
Chart	0.0001	(−0.0038, 0.0040)	0.0001	(−0.0035, 0.0038)	0.0048	**(0.0011, 0.0086)**
Cli	−0.0014	(−0.0096, 0.0069)	−0.0039	(−0.0109, 0.0030)	0.0038	(−0.0027, 0.0104)
Closure	0.0098	**(0.0075, 0.0122)**	0.0056	**(0.0032, 0.0081)**	0.0062	**(0.0036, 0.0087)**
Codec	0.0174	**(0.0118, 0.0231)**	0.0110	**(0.0054, 0.0168)**	0.0059	**(0.0010, 0.0109)**
Compress	0.0067	**(0.0032, 0.0102)**	0.0057	**(0.0023, 0.0092)**	0.0099	**(0.0060, 0.0138)**
Csv	0.0016	(−0.0052, 0.0086)	−0.0016	(−0.0080, 0.0048)	0.0065	**(0.0008, 0.0124)**
Gson	−0.0016	(−0.0062, 0.0030)	0.0001	(−0.0045, 0.0047)	0.0007	(−0.0038, 0.0053)
JacksonCore	0.0064	**(0.0031, 0.0098)**	0.0033	**(0.0001, 0.0066)**	0.0010	(−0.0021, 0.0042)
JacksonDatabind	−0.0028	**(−0.0049, −0.0008)**	−0.0047	**(−0.0072, −0.0022)**	−0.0025	**(−0.0046, −0.0004)**
JacksonXml	0.0469	**(0.0240, 0.0708)**	0.0473	**(0.0252, 0.0705)**	0.0343	**(0.0160, 0.0533)**
Jsoup	0.0032	**(0.0012, 0.0052)**	0.0017	(−0.0002, 0.0036)	0.0012	(−0.0006, 0.0031)
JxPath	0.0067	**(0.0027, 0.0109)**	0.0067	**(0.0022, 0.0111)**	0.0098	**(0.0053, 0.0145)**
Lang	0.0043	**(0.0028, 0.0059)**	0.0056	**(0.0040, 0.0072)**	0.0051	**(0.0035, 0.0067)**
Math	0.0047	**(0.0032, 0.0062)**	0.0059	**(0.0044, 0.0076)**	0.0055	**(0.0039, 0.0071)**
Mockito	−0.0012	(−0.0060, 0.0035)	0.0047	(−0.0011, 0.0106)	−0.0001	(−0.0041, 0.0040)
Time	0.0201	**(0.0141, 0.0264)**	0.0330	**(0.0237, 0.0424)**	0.0203	**(0.0138, 0.0270)**
Total*	0.0049	**(0.0043, 0.0055)**	0.0052	**(0.0046, 0.0060)**	0.0046	**(0.0039, 0.0053)**
Total	0.0044	**(0.0039, 0.0050)**	0.0044	**(0.0038, 0.0050)**	0.0040	**(0.0034, 0.0046)**

7.4 Use of Original vs. Clean Test Effectiveness Metrics

From RQ1 to RQ5, the analysis follows three steps shown in Figure 2: estimation of the confounding effect (Step 1), removal of the confounding effect (Step 2), and evaluation of test effectiveness metrics (Step 3). Specifically, for RQ1 to RQ3, we use the number of test cases as the measure of test suite size. RQ1, RQ2, and RQ3 follow these three steps in sequence. For RQ4, we analyze the confounding effect of test suite size using alternative test suite size measures. For RQ5, we conduct an analysis that uses code coverage measures to predict mutation scores. RQ4 and RQ5 each follow the three steps.

The first step is the estimation of the confounding effect. If the statistical test indicates a significant confounding effect, then the next two steps should be performed: (1) using the confounding removal method to obtain clean code coverage or mutation score (Step 2) and (2) using the clean code coverage or mutation score to analyze the true relationship between code coverage or mutation score and defect detection (Step 3). However, if the statistical test does not indicate a significant



(a) Mostly Inadequate Test Suites

(b) Adequate Test Suites

Fig. 8. Violin plots of probabilistic coupling achieved by statement coverage and mutation score.

confounding effect, there is no need to perform the following two steps, and the original code coverage or mutation score should be used instead.

7.5 Comparison with Probabilistic Coupling

Chen et al. [17] proposed PC to assess the ability of a test effectiveness metric in detecting actual faults. PC is calculated as $PC(f, g_i) = P(\text{defect } f | g_i \text{ is detected})$, where f represents an actual defect and g_i is a test goal (e.g., a statement, a mutant). For all possible test goals G (e.g., reached statements and covered mutants), $PC(f, G) = \max_{g_i \in G} PC(f, g_i)$, which is the maximum value of $PC(f, g_i)$ for all g_i in G.

However, our previous study [118] identifies several limitations of PC: (1) PC is limited to the detection of a single defect and is not directly applicable for detecting multiple defects. (2) PC is ideally calculated with adequate test suites that cover all possible test goals (e.g., reached statements and covered mutants). (3) PC is not directly applicable to branch coverage, which may contain an unreachable false branch with only the true branch being reachable, or vice versa.

The comparison of different test effectiveness metrics in this article contrasts with the use of PC. We perform comparisons by removing the confounding effect of test suite size. Additionally, the confounding model is suitable not only when defect detection is modeled as a dichotomous variable. In Section 7.3, we have included analysis using the mutation score as a proxy for defect detection, which is modeled as a continuous variable. This overcomes the limitation of PC, which is not directly applicable for detecting multiple defects. Furthermore, we generate test suites by satisfying different thresholds of code coverage and mutation score using greedy selection. The test suites generated are predominantly inadequate. Moreover, we analyze the capability of branch coverage in predicting test suite effectiveness.

Given the limitations of PC, we conduct experiments with test suites constructed based on statement coverage and mutation score. Initially, we use test suites constructed through greedy selection, resulting in 15,069 test suites based on statement coverage and 14,149 test suites based on mutation score, respectively, across projects that exhibit a significant confounding effect of test suite size. This involves 480 and 414 defects for statement coverage and mutation score, respectively. We calculate the average PC of test suites generated based on statement coverage and mutation score for each defect, respectively. This is calculated for each defect as the average of the maximum PC achieved by each test suite. Figure 8(a) presents the distribution of the average PC for each defect. The test suites used to calculate this PC are mostly inadequate. Clearly, the PC is generally low, with both the mean and the median being less than 0.5. The mutation score achieves a higher PC than statement coverage.

As it is ideal to calculate PC using adequate test suites, we further calculate the PC based on reduced test suites using $greedy_{sza_adq}$. These test suites are adequate, covering all possible test

goals, just as the unreduced test suites do. We calculate the PC among the 480 and 414 defects spanning across projects that exhibit a significant confounding effect of test suite size on the relationship between statement coverage and mutation score for defect detection, respectively. Since we perform test suite reduction five times for each defect, we calculate the average of the maximum PC achieved by each reduced test suite for each defect. Figure 8(b) presents the distribution of the average PC for each defect, calculated using adequate test suites. The average values of the PC achieved by statement coverage and mutation score across 480 defects and 414 defects are 0.46 and 0.85, respectively, with the mutation score achieving a much higher average PC than statement coverage. The results of PC are consistent with the results of the comparison of different test effectiveness metrics in Section 6.3 of this article.

7.6 Implications

We have undertaken a comprehensive study to delve into the potential impact of test suite size on assessing test suite effectiveness. Our findings yield valuable implications for both researchers and practitioners within the software testing community.

(1) *Employing statistical models to deconfound the effect of test suite size during the evaluation of test effectiveness metrics.* In literature, many studies [45, 58, 71, 77] have highlighted the correlation between test suite size and both test effectiveness metrics and defect detection capabilities. Consequently, when evaluating test effectiveness metrics, it becomes crucial to deal with the potential confounding effect of test suite size. Previous studies [45, 77] have utilized the stratification method, creating groups with similar test suite sizes for analysis. However, Chen et al. [17] argue against directly manipulating test suite size. They suggest that such manipulation not only deviates from actual test creation practices but could also result in misinterpretations of observed correlations, leading to misleading conclusions. In contrast to the stratification method, our study adopts the confounding model to examine the potentially confounding effect of test suite size and introduces a statistical deconfounding method to remove the influence of test suite size without directly manipulating it. An actionable insight is to employ the confounding model and the deconfounding method to examine and control the potentially confounding effect of test suite size in studies investigating the relationship between test effectiveness metrics and defect detection. In future studies, we recommend employing our methodology to evaluate existing and newly developed test suite effectiveness metrics. This approach will yield cleaned metrics, allowing for a more accurate examination of their genuine association with defect detection. By employing our deconfounding method, software testing community acquires the ability to explore the genuine value of test effectiveness metrics.

(2) *Mutation score exhibits a stronger genuine association with defect detection compared with both branch coverage and statement coverage.* Our study, after removing the confounding influence of test suite size, establishes a stronger correlation between mutation score and defect detection compared to branch coverage and statement coverage. This finding echoes earlier research [8, 51], emphasizing the more dependable relationship between mutation score and test suite effectiveness over code coverage metrics. What sets our study apart is our methodological approach, wherein we conducted comparisons while deconfounding the effect of test suite size. This approach ensures a more precise comparative analysis, as both code coverage and mutation scores remain unaffected by test suite size. This provides a clear and actionable insight into comparing the effectiveness of various test effectiveness metrics, free from the confounding influence of test suite size. Consequently, test suites with high mutation score should offer practical advantages in software testing. One obstacle in using mutation score is the associated high computational cost. Nevertheless, several studies [55, 73, 114, 117] have explored mutation reduction and predictive mutation testing to mitigate the high cost, providing lightweight solutions for computing mutation scores. Therefore, we recommend

employing mutation score to evaluate the effectiveness of test suites in defect detection in practical applications.

(3) *Test suite reduction could lead to a more cost-effective suite after incorporating an adjustment for* test suite size. The process of test suite reduction involves strategically selecting and eliminating redundant or overlapping test cases within a test suite. This systematic approach aims to optimize the suite by removing unnecessary elements while ensuring sufficient coverage. However, reducing the suite inevitably leads to a potential decrease in defect detection capability. The challenge lies in minimizing this loss while trimming the size. Our study indicates that, after incorporating an adjustment for test suite size, the resulting suite achieves a better balance between size reduction and the associated loss in defect detection. Consequently, this yields a more cost-effective test suite. The actionable insight derived from our study is to advocate for the use of the code coverage and mutation score, which incorporate an adjustment of test suite size, in coverage-based and mutation-based test suite reduction strategies. Considering this advantage, we recommend incorporating an adjustment for test suite size before conducting coverage-based and mutation-based test suite reductions in practice.

7.7 Threats to Validity

We discuss the construct validity, internal validity, and external validity of our study. Construct validity is concerned with whether the study accurately measures or represents the concept it claims to. Internal validity assesses whether the changes in the dependent variable are indeed caused by the manipulation of the independent variable, and not by other factors. External validity refers to the extent to which the results of the study can be generalized to other research settings, populations, and conditions.

7.7.1 Construct Validity. In this article, we measure code coverage and mutation score using Cobertura and Major, respectively. Both Cobertura and Major are widely used in previous studies [17, 35, 51, 90, 115], which mitigates the threat to the accuracy of the measurements. Equivalent mutants are mutants that are semantically equivalent to the original program. Since determining equivalent mutants is an undecidable problem, we, like previous studies [55, 118], do not specifically address equivalent mutants. We use the default settings of Major to calculate the mutation score, which do not consider mutants that are not covered by any test cases in the calculation.

Another threat to construct validity comes from the execution time of individual test cases. The execution time for each test case changes in different executions. To mitigate this threat, we execute each test case 10 times and use the average time of the tenth execution as the execution time for each test case.

7.7.2 Internal Validity. The main threat to internal validity is the inferential tests of the confounding effect. We primarily use percentile bootstrap to test the indirect effect of test effectiveness metrics on defect detection capability. To reduce this threat, we employ two other methods, i.e., the bias-corrected bootstrap and the Monte Carlo, to test the confounding effect of test suite size. There are consistent results of the three methods. We rely on a statistical model to test the significance of the confounding effect of test suite size. The results of this significance test depend on the data collected. To mitigate this threat, we have collected data from test suites constructed in 16 real-world projects. The aggregate analysis of all the test suites across these 16 projects indicates that test suite size does indeed have a confounding effect.

Another threat is the predictive model used to predict test suite effectiveness. We reduce this threat by utilizing seven different machine learning models, namely LR, DT, K-NN, NB, MLP, SVM, and RF.

7.7.3 External Validity. We conduct experiments with 16 Java projects and 565 real defects. These projects are open source, well maintained, and tested. The 16 projects come from a variety of application domains. For instance, the project Chart is a library for producing charts, and Jsoup is a Java library for HTML parsing. However, our results may not generalize to all projects in all application domains. To mitigate this threat, it is necessary to conduct the same study on other projects in other domains.

8 Conclusions and Future Work

In this article, we conduct an in-depth study on the potentially confounding effect of test suite size on the relationship between test effectiveness metrics and defect detection capability. Our analysis, which employs various statistical methods to examine the significance of the confounding effect, reveals that test suite size indeed has a significant confounding effect. This typically leads to an overestimation of the association between test effectiveness metrics and defect detection capability. The linear regression-based deconfounding method we use successfully eliminates the confounding effect of test suite size. Our work introduces a methodology to control the confounding effect of test suite size without the need for direct manipulation.

Having successfully removed the confounding effect of test suite size, we delve deeper into the real relationship between test effectiveness metrics and defect detection. Our analysis, based on size-effect-free code coverage and mutation score, uncovers that mutation score emerges as the strongest predictor for defect detection, with statement coverage ranking as the least influential. Once we incorporate an adjustment for test suite size, both coverage-based and mutation-based reduction methods strike a better balance between trimming size and the incurred loss in the ability to detect defects. Furthermore, this adjustment results in a slight improvement in the speed of defect detection for coverage-based test case prioritization.

In our forthcoming endeavors, first, we plan to broaden our examination of how test suite effectiveness metrics correlate with defect detection. This expansion involves extending beyond the widely acknowledged statement and branch coverage to encompass additional code coverage like MC/DC (Modified condition/decision coverage) coverage and method coverage. Second, we intend to implement our methodology in specific domains, such as web services, internet of things, and deep learning applications. This expansion will serve to validate and fine-tune our findings, providing a more comprehensive understanding of the usefulness of test effectiveness metrics in quantifying defect detection capability across diverse areas. Third, apart from the confounding effect, we plan to analyze the presence of other effects, such as the interaction effect between test effectiveness metrics and test suite size.

References

[1] Cobertura. Retrieved from https://cobertura.github.io/cobertura/

[2] Supplementary Material. Retrieved from https://figshare.com/articles/dataset/Supplementary_Material/24598707

[3] Alireza Aghamohammadi, Seyed-Hassan Mirian-Hosseinabadi, and Sajad Jalali. 2021. Statement frequency coverage: A code coverage criterion for assessing test suite effectiveness. *Information and Software Technology* 129 (2021), 106426.

[4] Josephine Akosa. 2017. Predictive accuracy: A misleading performance measure for highly imbalanced data. In *Proceedings of the 12th SAS Global Forum*, 1–4.

[5] Duane F. Alwin and Robert M. Hauser. 1975. The decomposition of effects in path analysis. *American Sociological Review* 40, 1 (1975), 37–47.

[6] James H. Andrews, Lionel C. Briand, Yvan Labiche, and Akbar Siami Namin. 2006. Using mutation analysis for assessing and comparing testing coverage criteria. *IEEE Transactions on Software Engineering* 32, 8 (2006), 608–624. DOI: https://doi.org/10.1109/TSE.2006.83

[7] George K. Baah, Andy Podgurski, and Mary Jean Harrold. 2011. Mitigating the confounding effects of program dependences for effective fault localization. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, 146–156. DOI: https://doi.org/10.1145/2025113.2025136

[8] Richard Baker and Ibrahim Habli. 2012. An empirical evaluation of mutation testing for improving the test quality of safety-critical software. *IEEE Transactions on Software Engineering* 39, 6 (2012), 787–805. DOI: https://doi.org/10.1109/TSE.2012.56

[9] Jennifer Black, Emanuel Melachrinoudis, and David Kaeli. 2004. Bi-criteria models for all-uses test suite reduction. In *Proceedings of the 26th International Conference on Software Engineering*. IEEE, 106–115. DOI: https://doi.org/10.1109/ICSE.2004.1317433

[10] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2329–2344.

[11] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2016. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 1032–1043.

[12] Marcel Böhme, László Szekeres, and Jonathan Metzman. 2022. On the reliability of coverage-based fuzzer benchmarking. In *Proceedings of the 44th International Conference on Software Engineering*, 1621–1633.

[13] Jacob Burnim and Koushik Sen. 2008. Heuristics for scalable dynamic test generation. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 443–446.

[14] Taejoon Byun, Vaibhav Sharma, Sanjai Rayadurgam, Stephen McCamant, and Mats P. E. Heimdahl. 2017. Toward rigorous object-code coverage criteria. In *2017 IEEE 28th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 328–338.

[15] Thierry Titcheu Chekam, Mike Papadakis, Yves Le Traon, and Mark Harman. 2017. An empirical study on mutation, statement and branch coverage fault revelation that avoids the unreliable clean program assumption. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 597–608. DOI: https://doi.org/10.1109/ICSE.2017.61

[16] Tsong Yueh Chen and Man Fai Lau. 1998. A new heuristic for test suite reduction. *Information and Software Technology* 40, 5–6 (1998), 347–354. DOI: https://doi.org/10.1016/S0950-5849(98)00050-0

[17] Yiqun T. Chen, Rahul Gopinath, Anita Tadakamalla, Michael D. Ernst, Reid Holmes, Gordon Fraser, Paul Ammann, and René Just. 2020. Revisiting the relationship between fault detection, test adequacy criteria, and test set size. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 237–249. DOI: https://doi.org/10.1145/3324884.3416667

[18] Zhichao Chen, Junjie Chen, Weijing Wang, Jianyi Zhou, Meng Wang, Xiang Chen, Shan Zhou, and Jianmin Wang. 2023. Exploring better black-box test case prioritization via log analysis. *ACM Transactions on Software Engineering and Methodology* 32, 3 (2023), 1–32.

[19] Hugh Coolican. 2009. Research methods and statistics in psychology. DOI: https://doi.org/10.4324/9780203769669

[20] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology* 171 (2024), 107468.

[21] Albert Danial. 2021. *Cloc: v1.92*. DOI: https://doi.org/10.5281/zenodo.5760077

[22] James Devasia, Hridayanand Goswami, Subitha Lakshminarayanan, Manju Rajaram, and Subathra Adithan. 2023. Deep learning classification of active tuberculosis lung zones wise manifestations using chest X-rays: A multi label approach. *Scientific Reports* 13, 1 (2023), 887.

[23] William Dickinson, David Leon, and A. Fodgurski. 2001. Finding failures by cluster analysis of execution profiles. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE '01)*. IEEE, 339–348.

[24] Khaled El Emam, Saïda Benlarbi, Nishith Goel, and Shesh N. Rai. 2001. The confounding effect of class size on the validity of object-oriented metrics. *IEEE Transactions on Software Engineering* 27, 7 (2001), 630–650. DOI: https://doi.org/10.1109/32.935855

[25] Mehran Farajollahi, Shima Saber Tahan, Abbasali Keshtkar, and Ove A. Peters. 2025. Confounders: A threat to the interpretation of research findings in endodontics. *International Endodontic Journal* 58, 3 (2025), 411–419.

[26] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, 416–419. DOI: https://doi.org/10.1145/2025113.2025179

[27] Gordon Fraser, Matt Staats, Phil McMinn, Andrea Arcuri, and Frank Padberg. 2013. Does automated white-box test generation really help software testers? In *Proceedings of the 2013 International Symposium on Software Testing and Analysis*, 291–301. DOI: https://doi.org/10.1145/2483760.2483774

[28] Gordon Fraser, Matt Staats, Phil McMinn, Andrea Arcuri, and Frank Padberg. 2015. Does automated unit test generation really help software testers? A controlled empirical study. *ACM Transactions on Software Engineering and Methodology* 24, 4 (2015), 1–49.

[29] Catherine O. Fritz, Peter E. Morris, and Jennifer J. Richler. 2012. Effect size estimates: Current use, calculations, and interpretation. *Journal of Experimental Psychology: General* 141, 1 (2012), 2.

[30] M. S. Fritz, A. B. Taylor, and D. P. Mackinnon.2012. Explanation of two anomalous results in statistical mediation analysis. *Multivariate Behavioral Research* 47, 1 (2012), 61–87.

[31] Gregory Gay. 2017. The fitness function for the job: Search-based generation of test suites that detect real faults. In *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 345–355. DOI: https://doi.org/10.1109/ICST.2017.38

[32] Milos Gligoric, Alex Groce, Chaoqiang Zhang, Rohan Sharma, Mohammad Amin Alipour, and Darko Marinov. 2013. Comparing non-adequate test suites using coverage criteria. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis*, 302–313. DOI: https://doi.org/10.1145/2483760.2483769

[33] Milos Gligoric, Alex Groce, Chaoqiang Zhang, Rohan Sharma, Mohammad Amin Alipour, and Darko Marinov. 2015. Guidelines for coverage-based comparisons of non-adequate test suites. *ACM Transactions on Software Engineering and Methodology* 24, 4 (2015), 1–33. DOI: https://doi.org/10.1145/2660767

[34] Yue Gong. 2023. *Meta-Learning with Differentiable Convex Optimization*. Technical Report. EasyChair.

[35] Rahul Gopinath, Carlos Jensen, and Alex Groce. 2014. Code coverage for suite evaluation by developers. In *Proceedings of the 36th International Conference on Software Engineering*, 72–82. DOI: https://doi.org/10.1145/2568225.2568278

[36] Siqi Gu, Chunrong Fang, Quanjun Zhang, Fangyuan Tian, and Zhenyu Chen. 2024. Testart: Improving llm-based unit test via co-evolution of automated generation and repair iteration. arXiv:2408.03095. Retrieved from https://arxiv.org/abs/2408.03095

[37] M. Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. 1993. A methodology for controlling the size of a test suite. *ACM Transactions on Software Engineering and Methodology* 2, 3 (1993), 270–285. DOI: https://doi.org/10.1109/ICSM.1990.131378

[38] Lisa A. Harvey. 2020. Relationships, associations, risk factors and correlations: Nebulous phrases without obvious clinical implications. *Spinal Cord* 58, 1 (2020), 1–2.

[39] Mohammad Mahdi Hassan and James H. Andrews. 2013. Comparing multi-point stride coverage and dataflow coverage. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 172–181. DOI: https://doi.org/10.1109/ICSE.2013.6606563

[40] Andrew F. Hayes. 2017. *Introduction to Mediation, Moderation, and Conditional Process Analysis: A Regression-Based Approach*. Guilford Publications.

[41] Andrew F. Hayes and Michael Scharkow. 2013. The relative trustworthiness of inferential tests of the indirect effect in statistical mediation analysis: Does method really matter? *Psychological Science* 24, 10 (2013), 1918–1927.

[42] Hadi Hemmati. 2015. How effective are code coverage criteria? In *2015 IEEE International Conference on Software Quality, Reliability and Security*. IEEE, 151–156.

[43] Shin Hong, Matt Staats, Jaemin Ahn, Moonzoo Kim, and Gregg Rothermel. 2013. The impact of concurrent coverage metrics on testing effectiveness. In *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*. IEEE, 232–241.

[44] Han Huang, Fangqing Liu, Xiaoyan Zhuo, and Zhifeng Hao. 2017. Differential evolution based on self-adaptive fitness function for automated test case generation. *IEEE Computational Intelligence Magazine* 12, 2 (2017), 46–55.

[45] Laura Inozemtseva and Reid Holmes. 2014. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering*, 435–445. DOI: https://doi.org/10.1145/2568225.2568271

[46] K. J. Jager, C. Zoccali, A. Macleod, and F. W. Dekker. 2008. Confounding: What it is and how to deal with it. *Kidney International* 73, 3 (2008), 256–260.

[47] Yue Jia and Mark Harman. 2010. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering* 37, 5 (2010), 649–678.

[48] Mayank Juneja, Andrea Vedaldi, C. V. Jawahar, and Andrew Zisserman. 2013. Blocks that shout: Distinctive parts for scene classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 923–930.

[49] René Just. 2014. The major mutation framework: Efficient and scalable mutation analysis for java. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 433–436.

[50] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 437–440. DOI: https://doi.org/10.1145/2610384.2628055

[51] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser.2014. Are mutants a valid substitute for real faults in software testing?. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 654–665. DOI: https://doi.org/10.1109/ICST.2011.46

[52] William Kevin Kelly and Susan Halabi. 2018. *Oncology Clinical Trials: successful Design, Conduct, and Analysis*. Springer Publishing Company.

[53] Saif Ur Rehman Khan, Sai Peck Lee, Nadeem Javaid, and Wadood Abdul. 2018. A systematic review on test suite reduction: Approaches, experiment's quality evaluation, and guidelines. *IEEE Access* 6 (2018), 11816–11841. DOI: https://doi.org/10.1109/ACCESS.2018.2809600

[54] Muhammad Khatibsyarbini, Mohd Adham Isa, Dayang N. A. Jawawi, and Rooster Tumeng. 2018. Test case prioritization approaches in regression testing: A systematic literature review. *Information and Software Technology* 93 (2018), 74–93. DOI: https://doi.org/10.1016/j.infsof.2017.08.014

[55] Jinhan Kim, Juyoung Jeon, Shin Hong, and Shin Yoo. 2022. Predictive mutation analysis via the natural language channel in source code. *ACM Transactions on Software Engineering and Methodology* 31, 4 (2022), 1–27.

[56] Mingwan Kim, Neunghoe Kim, and Hoh Peter In. 2020. Investigating the relationship between mutants and real faults with respect to mutated code. *International Journal of Software Engineering and Knowledge Engineering* 30, 8 (2020), 1119–1137. DOI: https://doi.org/10.1142/S021819402050028X

[57] Pavneet Singh Kochhar, David Lo, Julia Lawall, and Nachiappan Nagappan. 2017. Code coverage and postrelease defects: A large-scale study on open source projects. *IEEE Transactions on Reliability* 66, 4 (2017), 1213–1228.

[58] Pavneet Singh Kochhar, Ferdian Thung, and David Lo. 2015. Code coverage and test suite effectiveness: Empirical study with real bugs in large systems. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 560–564. DOI: https://doi.org/10.1109/SANER.2015.7081877

[59] Demetrios N. Kyriacou and Roger J. Lewis. 2016. Confounding by indication in clinical research. *JAMA* 316, 17 (2016), 1818–1819.

[60] Caroline Lemieux and Koushik Sen. 2018. Fairfuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 475–485.

[61] Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. 2023. Taco: Topics in algorithmic code generation dataset. arXiv:2312.14852. Retrieved from https://arxiv.org/abs/2312.14852

[62] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.

[63] Ying Li, Hojjat Salmasian, Santiago Vilar, Herbert Chase, Carol Friedman, and Ying Wei. 2014. A method for controlling complex confounding effects in the detection of adverse drug reactions using electronic health records. *Journal of the American Medical Informatics Association* 21, 2 (2014), 308–314.

[64] Yuekang Li, Yinxing Xue, Hongxu Chen, Xiuheng Wu, Cen Zhang, Xiaofei Xie, Haijun Wang, and Yang Liu. 2019. Cerebro: Context-aware adaptive fuzzing for effective vulnerability detection. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 533–544.

[65] Yiling Lou, Dan Hao, and Lu Zhang. 2015. Mutation-based test-case prioritization in software evolution. In *2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 46–57. DOI: https://doi.org/10.1109/ISSRE.2015.7381798

[66] David P. MacKinnon. 2012. *Introduction to Statistical Mediation Analysis*. Routledge.

[67] David P. MacKinnon, Jennifer L. Krull, and Chondra M. Lockwood. 2000. Equivalence of the mediation, confounding and suppression effect. *Prevention Science* 1 (2000), 173–181.

[68] David P. MacKinnon, Chondra M. Lockwood, and Jason Williams. 2004. Confidence limits for the indirect effect: Distribution of the product and resampling methods. *Multivariate Behavioral Research* 39, 1 (2004), 99–128.

[69] Abla Mehio-Sibai, Manning Feinleib, Tarek A. Sibai, and Haroutune K. Armenian. 2005. A positive or a negative confounding variable? A simple teaching aid for clinicians and students. *Annals of Epidemiology* 15, 6 (2005), 421–423.

[70] Dipak Kumar and Mitra Abdullah H. Baqui. 2023. Bias, confounding, and effect modifier. In *Statistical Approaches for Epidemiology: From Concept to Application*. Springer, 169–182.

[71] Akbar Siami Namin and James H. Andrews. 2009. The influence of size and coverage on test suite effectiveness. In *Proceedings of the 18th International Symposium on Software Testing and Analysis*, 57–68. DOI: https://doi.org/10.1145/1572272.1572280

[72] Ioannis Ntzoufras. 2011. *Bayesian Modeling Using WinBUGS*. John Wiley & Sons.

[73] A. Jefferson Offutt and Roland H. Untch. 2001. Mutation 2000: Uniting the orthogonal. *Mutation Testing for the New Century* (2001), 34–44. DOI: https://doi.org/10.1007/978-1-4757-5939-6_7

[74] Jeff Offutt, Jie Pan, and Jeffrey M. Voas. 1995. Procedures for reducing the size of coverage-based test sets. In *Proceedings of the 12th International Conference on Testing Computer Software*. ACM Press, New York, 111–123.

[75] Sinan Ozdemir and Divya Susarla. 2018. *Feature Engineering Made Easy: identify unique features from Your Dataset in Order to Build Powerful Machine Learning Systems*. Packt Publishing Ltd.

[76] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: Feedback-directed random testing for java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion*, 815–816. DOI: https://doi.org/10.1145/1297846.1297902

[77] Mike Papadakis, Donghwan Shin, Shin Yoo, and Doo-Hwan Bae. 2018. Are mutation scores correlated with real fault detection? A large scale empirical study on the relationship between mutants and real faults. In *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*. IEEE, 537–548. DOI: https://doi.org/10.1145/3180155.3180183

[78] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg , et al. 2011. Scikit-learn: Machine learning in python. *The Journal of Machine Learning Research* 12 (2011), 2825–2830.
[79] Jean Petrić, Tracy Hall, and David Bowes. 2020. Which software faults are tests not detecting? In *Proceedings of the Evaluation and Assessment in Software Engineering*, 160–169. DOI: https://doi.org/10.1145/3383219.3383236
[80] Van-Thuan Pham, Marcel Böhme, Andrew E. Santosa, Alexandru Răzvan Căciulescu, and Abhik Roychoudhury. 2019. Smart greybox fuzzing. *IEEE Transactions on Software Engineering* 47, 9 (2019), 1980–1997.
[81] Mohamad Amin Pourhoseingholi, Ahmad Reza Baghestani, and Mohsen Vahedi. 2012. How to control confounding effects by statistical analysis. *Gastroenterology and Hepatology from Bed to Bench* 5, 2 (2012), 79.
[82] Kristopher J. Preacher and Andrew F. Hayes. 2008. Asymptotic and resampling strategies for assessing and comparing indirect effects in multiple mediator models. *Behavior Research Methods* 40, 3 (2008), 879–891.
[83] Ajitha Rajan, Michael W. Whalen, and Mats P. E. Heimdahl. 2008. The effect of program and model structure on MC/DC test adequacy coverage. In *Proceedings of the 30th International Conference on Software Engineering*, 161–170.
[84] Judith J. M. Rijnhart, Jos W. R. Twisk, Iris Eekhout, and Martijn W. Heymans. 2019. Comparison of logistic-regression based methods for simple mediation analysis with a dichotomous outcome variable. *BMC Medical Research Methodology* 19 (2019), 1–10.
[85] José Miguel Rojas, Thomas D. White, Benjamin S. Clegg, and Gordon Fraser. 2017. Code defenders: Crowdsourcing effective tests and subtle mutants with a mutation testing game. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 677–688.
[86] Gregg Rothermel, Mary Jean Harrold, Jeffery Ostrin, and Christie Hong. 1998. An empirical study of the effects of minimization on the fault detection capabilities of test suites. In *Proceedings of the International Conference on Software Maintenance (Cat. No. 98CB36272)*. IEEE, 34–43. DOI: https://doi.org/10.1109/ICSM.1998.738487
[87] Gregg Rothermel, Mary Jean Harrold, Jeffery Von Ronne, and Christie Hong. 2002. Empirical studies of test-suite reduction. *Software Testing, Verification and Reliability* 12, 4 (2002), 219–249. DOI: https://doi.org/10.1002/stvr.256
[88] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. 2001. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering* 27, 10 (2001), 929–948. DOI: https://doi.org/10.1109/32.962562
[89] Dharmashree Satyarup, Manish Kumar, Radha Prasanna Dalai, and Karishma R. Rathor. 2020. Confounding and effect modification in research. *Indian Journal of Forensic Medicine and Toxicology* 14, 4 (2020), 8394–8400.
[90] Amanda Schwartz, Daniel Puckett, Ying Meng, and Gregory Gay. 2018. Investigating faults missed by test suites achieving high code coverage. *Journal of Systems and Software* 144 (2018), 106–120. DOI: https://doi.org/10.1016/j.jss.2018.06.024
[91] Hyunmin Seo and Sunghun Kim. 2014. How we get there: A context-guided search strategy in concolic testing. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 413–424.
[92] Domenico Serra, Giovanni Grano, Fabio Palomba, Filomena Ferrucci, Harald C. Gall, and Alberto Bacchelli. 2019. On the effectiveness of manual and automatic unit test generation: Ten years later. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 121–125. DOI: https://doi.org/10.1109/MSR.2019.00028
[93] Dongdong She, Kexin Pei, Dave Epstein, Junfeng Yang, Baishakhi Ray, and Suman Jana. 2019. Neuzz: Efficient fuzzing with neural program smoothing. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 803–817.
[94] August Shi, Alex Gyori, Milos Gligoric, Andrey Zaytsev, and Darko Marinov. 2014. Balancing trade-offs in test-suite reduction. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 246–256. DOI: https://doi.org/10.1145/2635868.2635921
[95] August Shi, Alex Gyori, Suleman Mahmood, Peiyuan Zhao, and Darko Marinov. 2018. Evaluating test-suite reduction in real software evolution. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 84–94. DOI: https://doi.org/10.1145/3213846.3213875
[96] Donghwan Shin, Shin Yoo, and Doo-Hwan Bae. 2017. A theoretical and empirical study of diversity-aware mutation adequacy criterion. *IEEE Transactions on Software Engineering* 44, 10 (2017), 914–931.
[97] Donghwan Shin, Shin Yoo, Mike Papadakis, and Doo-Hwan Bae. 2019. Empirical evaluation of mutation-based test case prioritization techniques. *Software Testing, Verification and Reliability* 29, 1–2 (2019), e1695. DOI: https://doi.org/10.1002/stvr.1695
[98] Andrea C. Skelly, Joseph R. Dettori, and Erika D. Brodt. 2012. Assessing bias: The importance of considering confounding. *Evidence-Based Spine-Care Journal* 3, 1 (2012), 9–12.
[99] Jake Snell, Kevin Swersky, and Richard Zemel. 2017. Prototypical networks for few-shot learning. In Proceedings of the 31st International Conference on Neural Information Processing Systems. 4080–4090. .
[100] Michael E. Sobel. 1982. Asymptotic confidence intervals for indirect effects in structural equation models. *Sociological Methodology* 13 (1982), 290–312.
[101] Michael E. Sobel. 1987. Direct and indirect effects in linear structural equation models. *Sociological Methods and Research* 16, 1 (1987), 155–176.

[102] Amitabh Srivastava and Jay Thiagarajan. 2002. Effectively prioritizing tests in development environment. In *Proceedings of the 2002 ACM SIGSOFT International Symposium on Software Testing and Analysis*, 97–106. DOI: https://doi.org/10.1145/566171.566187

[103] Ting Su, Geguang Pu, Bin Fang, Jifeng He, Jun Yan, Siyuan Jiang, and Jianjun Zhao. 2014. Automated coverage-driven test data generation using dynamic symbolic execution. In *2014 8th International Conference on Software Security and Reliability (SERE)*. IEEE, 98–107.

[104] Moyses Szklo and F. Javier Nieto. 2014. *Epidemiology: Beyond the Basics*. Jones & Bartlett Publishers.

[105] K. J. Van Stralen, F. W. Dekker, C. Zoccali, and K. J. Jager. 2010. Confounding. *Nephron Clinical Practice* 116, 2 (2010), c143–c147.

[106] Jeffrey M. Voas. 1992. PIE: A dynamic failure-based technique. *IEEE Transactions on Software Engineering* 18, 8 (1992), 717.

[107] Yi Wei, Bertrand Meyer, and Manuel Oriol. 2012. Is branch coverage a good measure of testing effectiveness? In *Empirical Software Engineering and Verification: International Summer Schools (LASER 2008-2010)*, 194–212.

[108] W. Eric Wong, Joseph R. Horgan, Saul London, and Aditya P. Mathur. 1994. Effect of test set size and block coverage on the fault detection effectiveness. In *Proceedings of 1994 IEEE International Symposium on Software Reliability Engineering*. IEEE, 230–238. DOI: https://doi.org/10.1109/ISSRE.1994.341379

[109] W. Eric Wong, Joseph R. Horgan, Saul London, and Aditya P. Mathur. 1998. Effect of test set minimization on fault detection effectiveness. *Software: Practice and Experience* 28, 4 (1998), 347–369. DOI: https://doi.org/10.1145/225014.225018

[110] W. Eric Wong, Joseph Robert Horgan, Aditya P. Mathur, and Alberto Pasquini. 1997. Test set size minimization and fault detection effectiveness: A case study in a space application. In *Proceedings 21st Annual International Computer Software and Applications Conference (COMPSAC '97)*. IEEE, 522–528. DOI: https://doi.org/10.1109/CMPSAC.1997.625062

[111] Amy Yang. 2016. The Impact of Other Factors: Confounding, Mediation, and Effect Modification. DOI: https://doi.org/10.18131/G34897

[112] Ahmadreza Saboor Yaraghi, Mojtaba Bagherzadeh, Nafiseh Kahani, and Lionel C. Briand. 2022. Scalable and accurate test case prioritization in continuous integration contexts. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1615–1639.

[113] Ying Yuan and David P. MacKinnon. 2009. Bayesian mediation analysis. *Psychological Methods* 14, 4 (2009), 301.

[114] Jie Zhang, Lingming Zhang, Mark Harman, Dan Hao, Yue Jia, and Lu Zhang. 2018. Predictive mutation testing. *IEEE Transactions on Software Engineering* 45, 9 (2018), 898–918. DOI: https://doi.org/10.1109/TSE.2018.2809496

[115] Jie M. Zhang, Lingming Zhang, Dan Hao, Meng Wang, and Lu Zhang. 2019. Do pseudo test suites lead to inflated correlation in measuring test effectiveness? In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 252–263. DOI: https://doi.org/10.1109/ICST.2019.00033

[116] Lingming Zhang, Darko Marinov, Lu Zhang, and Sarfraz Khurshid. 2011. An empirical study of junit test-suite reduction. In *2011 IEEE 22nd International Symposium on Software Reliability Engineering*. IEEE, 170–179. DOI: https://doi.org/10.1109/ISSRE.2011.26

[117] Peng Zhang, Yanhui Li, Wanwangying Ma, Yibiao Yang, Lin Chen, Hongmin Lu, Yuming Zhou, and Baowen Xu. 2020. Cbua: A probabilistic, predictive, and practical approach for evaluating test suite effectiveness. *IEEE Transactions on Software Engineering* 48, 3 (2020), 1067–1096. DOI: https://doi.org/10.1109/TSE.2020.3010361

[118] Peng Zhang, Yang Wang, Xutong Liu, Zeyu Lu, Yibiao Yang, Yanhui Li, Lin Chen, Ziyuan Wang, Chang-Ai Sun, Xiao Yu, et al. 2023. Assessing effectiveness of test suites: What do we know and what should we do? *ACM Transactions on Software Engineering and Methodology* 33, 4 (2023), 1–32.

[119] Yuming Zhou, Hareton Leung, and Baowen Xu. 2009. Examining the potentially confounding effect of class size on the associations between object-oriented metrics and change-proneness. *IEEE Transactions on Software Engineering* 35, 5 (2009), 607–623. DOI: https://doi.org/10.1109/TSE.2009.32

[120] Yuming Zhou, Baowen Xu, Hareton Leung, and Lin Chen. 2014. An in-depth study of the potentially confounding effect of class size in fault prediction. *ACM Transactions on Software Engineering and Methodology* 23, 1 (2014), 1–51. DOI: https://doi.org/10.1145/2556777

Received 30 November 2023; revised 28 March 2025; accepted 7 July 2025